PH3120 - Computational Physics Laboratory I

CPL109

Chaos in Continuous Systems

G. W. T. Senesh

Reg NO: - 2022s19694

Index: - s16867

Contents

1. Introduction

Many physical systems are governed by higher-order differential equations, which can be converted into systems of first-order equations to make analysis easier. These systems are often explored by solving them numerically and observing their behavior in phase space—a graphical representation where each point reflects a specific system state. The full set of trajectories in this space forms the system's flow, which varies depending on initial conditions.

In some systems, different starting points produce different outcomes, while others tend to evolve toward common long-term behaviors known as attractors. These attractors might be fixed points, recurring loops (limit cycles), or more complex forms. A limit cycle represents periodic motion and draws in nearby paths, whereas unstable points like saddle nodes push trajectories away in some directions while pulling them in others.

In two-dimensional systems, dynamics such as sources, sinks, saddles, and limit cycles are often observed. However, three-dimensional systems can display much more intricate and unpredictable behaviors, including chaos. In chaotic systems, even the smallest changes in starting conditions can result in drastically different outcomes—a trait known as sensitivity to initial conditions.

Studying these flows helps reveal the structure and evolution of both predictable and chaotic systems over time.

2. Methods, Results and Discussion

2.1. Exercise 1(Simple pendulum)

2.2.1 Objective

The main objective of this is to analyze and visualize the behavior of a simple pendulum using its differential equations. It involves Solve the second order different equation with special initial conditions. Plot position and velocity vs time and plot phase space diagram. Essentially, the goal is to understand the dynamics of a simple pendulum through analytical solutions and various graphical representations, including time-domain plots and phase space diagrams, and to see how initial conditions and the natural frequency ω affect its behavior.

2.2.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 100)
omega = 2
x1 = 0.5 * np.sin(omega * t)
x2 = np.cos(omega * t)

# Plot 1: Position and Velocity vs Time
plt.figure(1)
plt.plot(t, x1, label='Position (x1)')
plt.plot(t, x2, label='Velocity (x2)')
plt.xlabel('Time (t)')
plt.ylabel('Value')
plt.title('Position and Velocity vs Time')
plt.legend()
plt.grid()

# Plot 2: Phase Space Diagram
plt.figure(2)
plt.plot(x1, x2)
plt.xlabel('Position (x1)')
plt.ylabel('Velocity (x2)')
plt.title('Phase Space Diagram')
plt.grid()

# Plot 3: Flow Map for different omega values
omegas = [1, 2, 3]
x = np.arange(-4, 4.1, 0.6)
y = np.arange(-6, 6.1, 0.8)
xv, yv = np.meshgrid(x, y)

for w in omegas:
    dx = yv
    dy = -w**2 * xv
    mag = np.sqrt(dx**2 + dy**2)
    dxu = dx / mag
    dyu = dy / mag
```

```
plt.figure(3)
plt.quiver(xv, yv, dxu, dyu, scale=1, scale_units='xy', angles='xy', width=0.004, color='b')
plt.xlabel('x')
plt.ylabel('y')
plt.title(f'Flow map for Simple Harmonic Motion (ω = {w})')
plt.grid()
plt.show()
```

Figure 2.1.1 : Code for Plot position and velocity, Plot the phase space diagram and plot the flow map for simple pendulum

2.2.3 How it works

Above python code uses Numpy and Matplotlib to simulate a simple pendulum. It defines time, position and velocity for omega = 2. Plot 1 graphs position and velocity versus time. Plot 2 creates a phase space diagram of position vs velocity. It showing an ellipse. Plot 3 generates flow maps for different omega values of 1,2 and 3 using a quiver plot, where $dx/dt=y$ and $dy/dt=-\omega^2 x$, illustrating vector fields. Each plot includes labels, titles, and grids for clarity.

2.2.4 Results

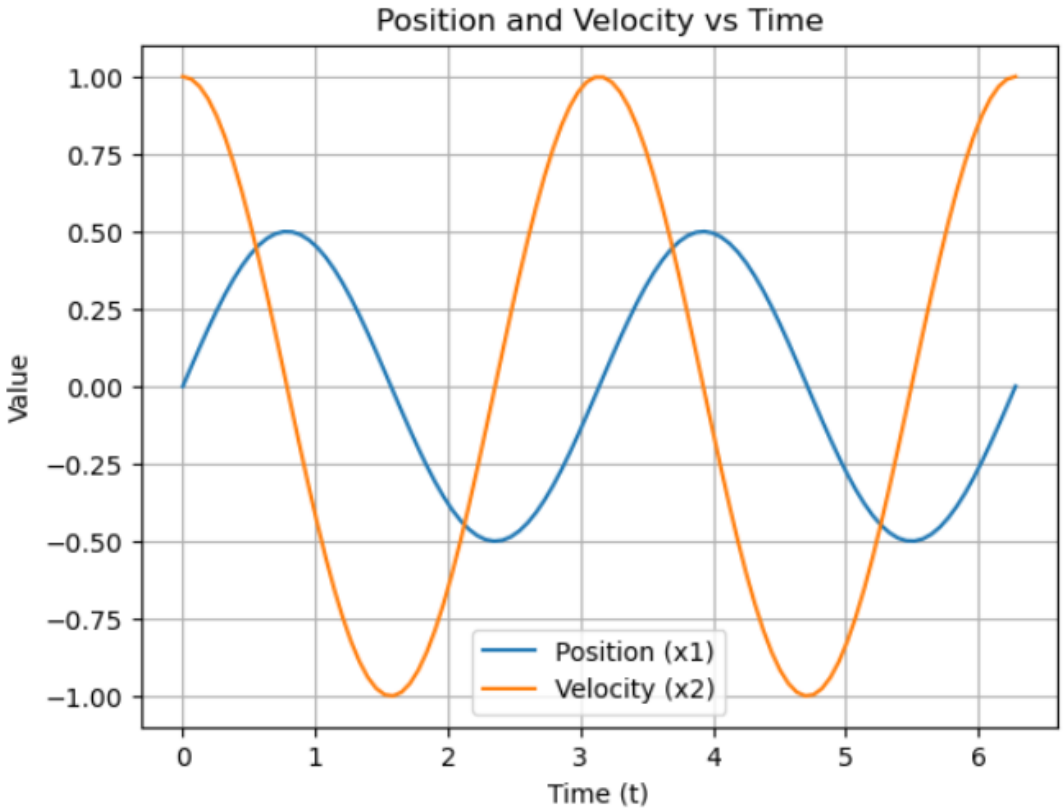

Figure 2.1.2: Plot of position and velocity versus time

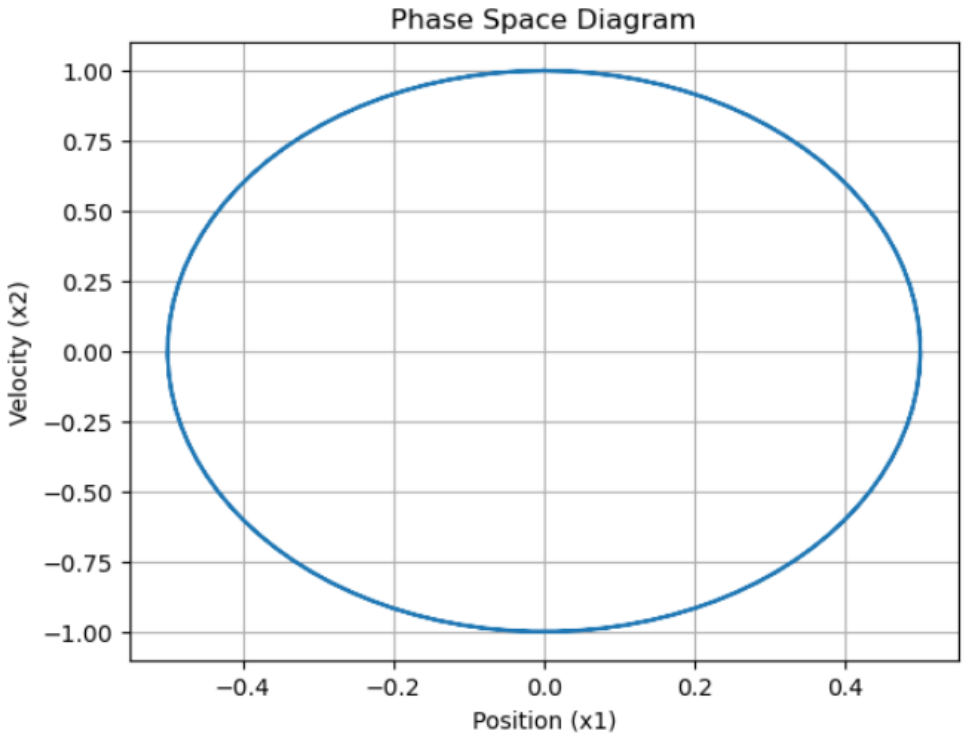

Figure 2.1.3: a phase space diagram of position vs velocity

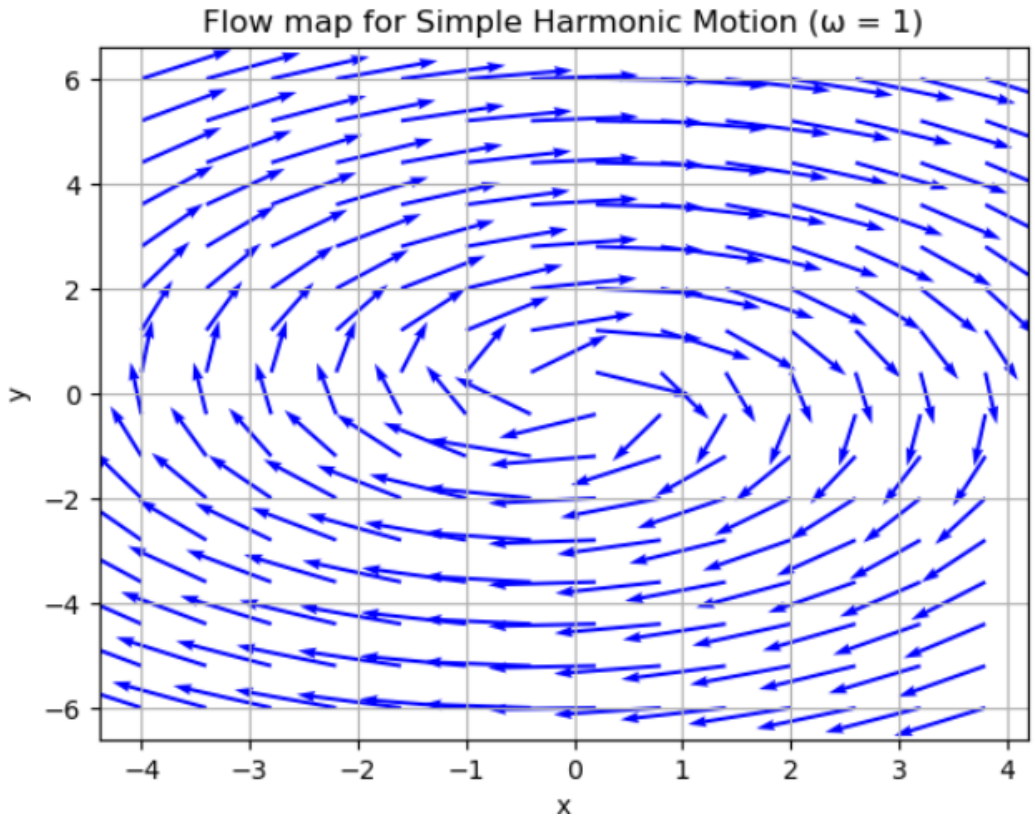

Figure 2.1.4: Flow map for Omega = 1

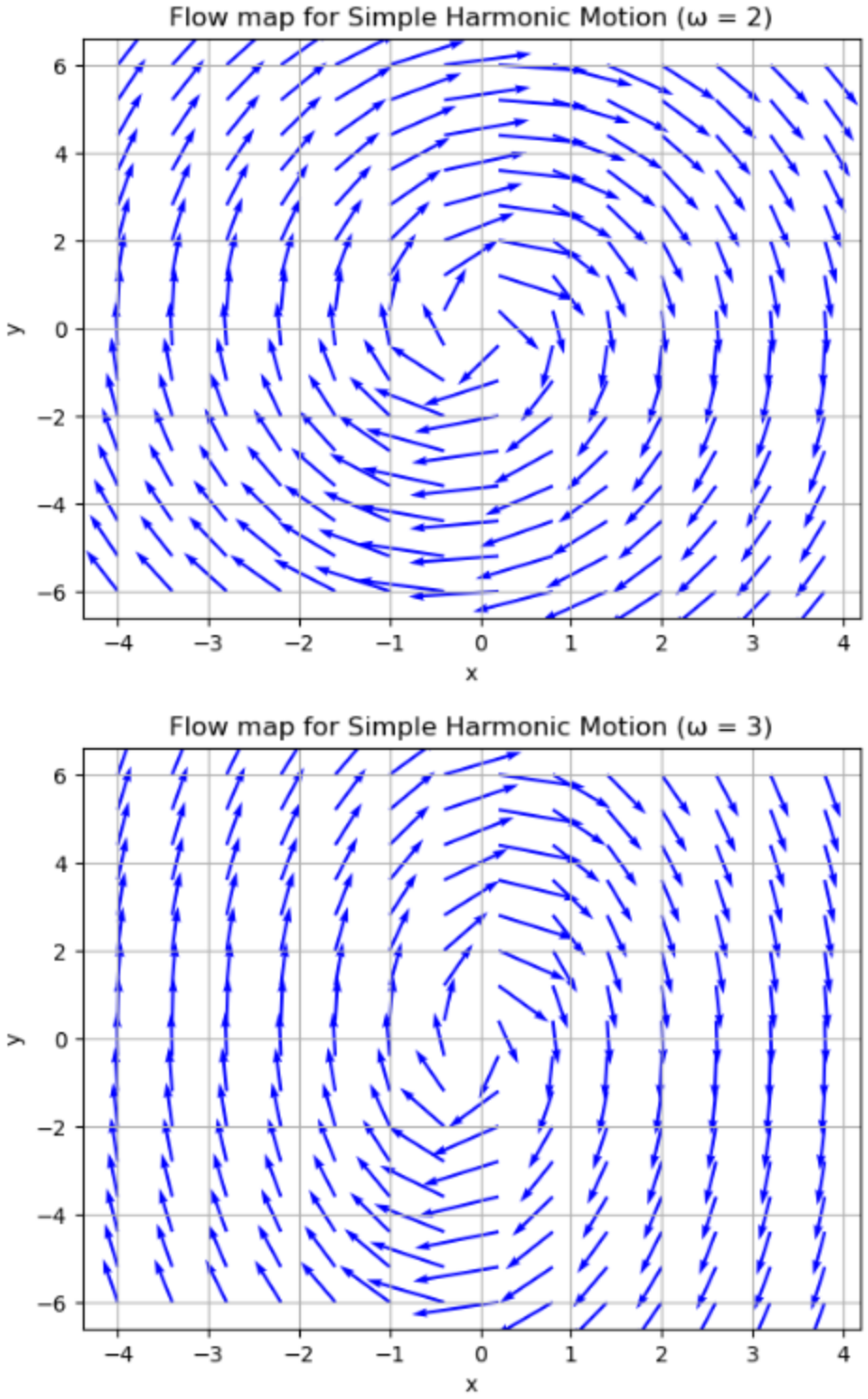

Figure 2.1.5: Flow map for Omega = 2 and Omega = 3

2.2.5 Discussion

The analysis of a simple pendulum, governed by $\ddot{x} = -\omega^2 x$, reveals its elegant dynamics. For $\omega = 2$ and initial conditions [0, 1], the pendulum exhibits harmonic motion, with position $x1(t) = 0.5\sin(2t)$ and velocity $x2(t) = \cos(2t)$.

The phase space diagram, plotting velocity against position, forms a characteristic ellipse. This elliptical trajectory is a signature of the pendulum's periodic behavior and energy conservation. Different initial conditions simply alter the starting point on this ellipse, but the fundamental shape and closed-loop nature remain constant, reflecting the system's return to its original state.

Flow maps, generated for $\omega = 1,2$, and 3, visually depict the vector fields in the phase space. These maps show the instantaneous direction and "flow" of the system. As ω increases, the ellipses in the flow map become more concentrated around the origin, indicating faster oscillations and higher velocities for given displacements. This demonstrates how frequency scales the size and speed of the phase space orbits. Collectively, these graphical representations, particularly the closed trajectories, underscore the simple pendulum's conservative nature, where mechanical energy is continually conserved, driving its perpetual, rhythmic motion.

2.2. Exercise 2 (Damped pendulum)

2.2.1 Objective

The main objective in this exercise is to analyze the behavior of a damped pendulum using numerical methods. This involves transferring the second order differential equation of the damped pendulum into a system of two first order different equations. Then Solving the first order equation numerically and plotting position and velocity using python code and plotting phase diagrams for different values of Alpha. Then modify existing code for create flow diagram.

2.2.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

omega = 2.0
alpha_values = [0.1, 0.5, 1.0, 2.0]

for alpha in alpha_values:
    t = np.linspace(0, 20, 1000)
    x0, v0 = 1.0, 0.0
    x = np.zeros_like(t)
    v = np.zeros_like(t)
    x[0], v[0] = x0, v0

    dt = t[1] - t[0]
    for i in range(1, len(t)):
        x[i] = x[i-1] + v[i-1] * dt
        v[i] = v[i-1] + (-omega**2 * x[i-1] - alpha * v[i-1]) * dt

    # Plot position and velocity vs time
    plt.figure()
    plt.plot(t, x, label='Position (x)')
    plt.plot(t, v, label='Velocity (v)')
    plt.xlabel('Time (t)')
    plt.ylabel('Value')
    plt.title(f'Position and Velocity vs Time (α = {alpha})')
    plt.legend()
    plt.grid()

    # Phase space diagram
    plt.figure()
    plt.plot(x, v)
    plt.xlabel('Position (x)')
    plt.ylabel('Velocity (v)')
    plt.title(f'Phase Space Diagram (α = {alpha})')
    plt.grid()
    plt.axis('equal')
```

```
# Flow diagram for alpha = 1, omega = 1
omega_flow = 1.0
alpha_flow = 1.0
x = np.linspace(-2, 2, 20)
y = np.linspace(-2, 2, 20)
X, Y = np.meshgrid(x, y)
dX = Y
dY = -omega_flow**2 * X - alpha_flow * Y
M = np.hypot(dX, dY)
dX = dX / M
dY = dY / M

plt.figure()
plt.quiver(X, Y, dX, dY, scale=50, color='b')
plt.xlabel('x')
plt.ylabel('v')
plt.title('Flow Diagram (ω = 1, α = 1)')
plt.grid()

x_eq, v_eq = 0, 0
print(f"Equilibrium point: ({x_eq}, {v_eq})")
A = np.array([[0, 1], [-omega_flow**2, -alpha_flow]])
eigenvalues = np.linalg.eigvals(A)
print(f"Eigenvalues: {eigenvalues}")

plt.show()
```

Figure 2.2.1 : Code for Plot position and velocity, Plot the phase space diagram and plot the flow map for damped oscillator

2.2.3 How it works

The Python code simulates a damped pendulum using NumPy and Matplotlib, modeling the equation $\ddot{x}=-\omega^2x-\alpha\dot{x}$ two first-order differential equations: $dx/dt = v$ and $dv/dt = -\omega^2x - \alpha v$, where $\omega=2.0$ is the natural frequency and α is the damping coefficient (varied as 0.1, 0.5, 1.0, 2.0).For each α, it creates a time array from 0 to 20 seconds with 1000 points, initializing position x and velocity v with starting values (1.0, 0.0). The Euler method numerically solves these equations iteratively, updating x and v based on the differential equations with a small-time step dt.

2.2.4 Results

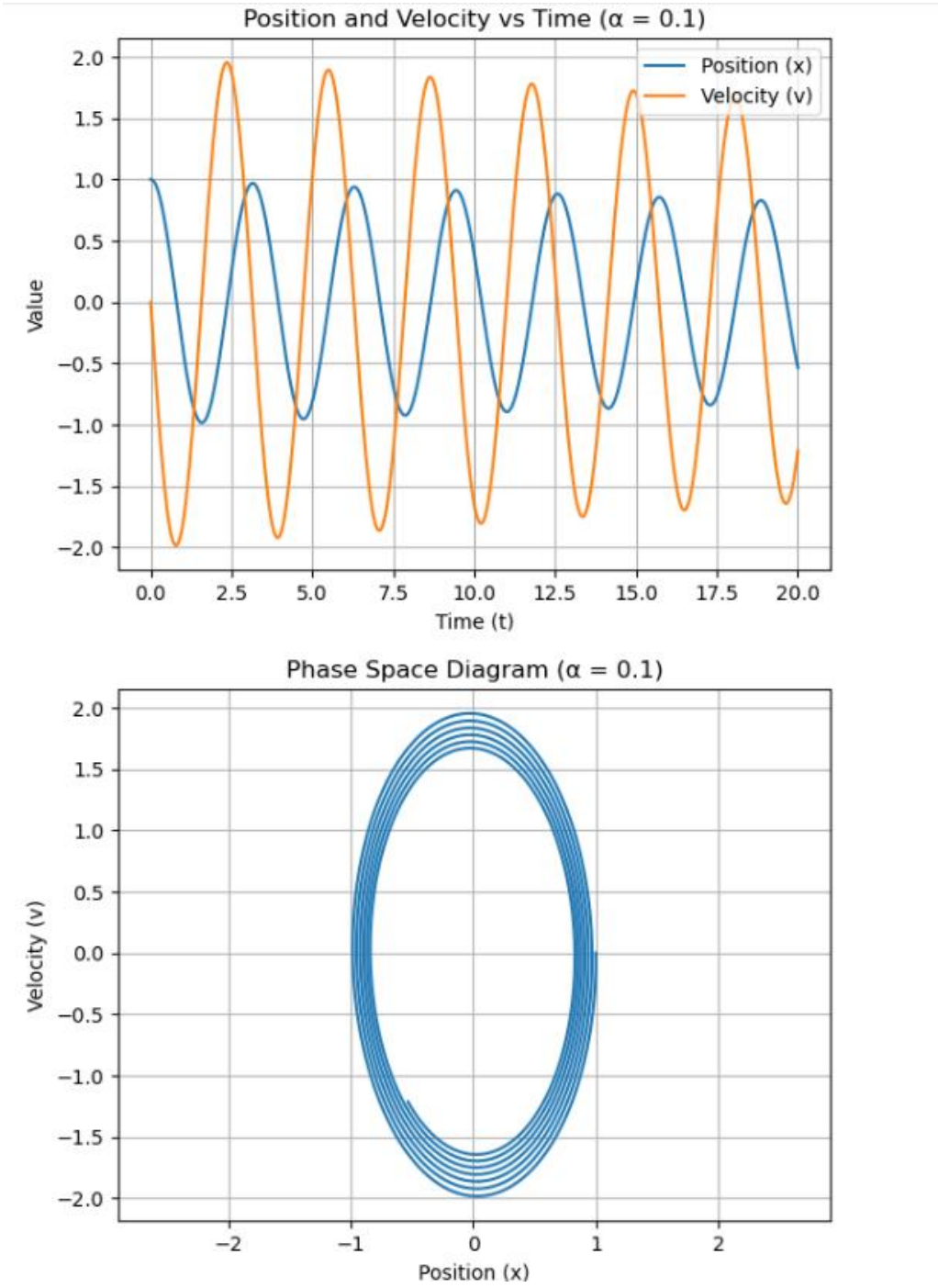

Figure 2.2.2 : Position and Velocity vs Time, Phase Space digram for α = 0.1

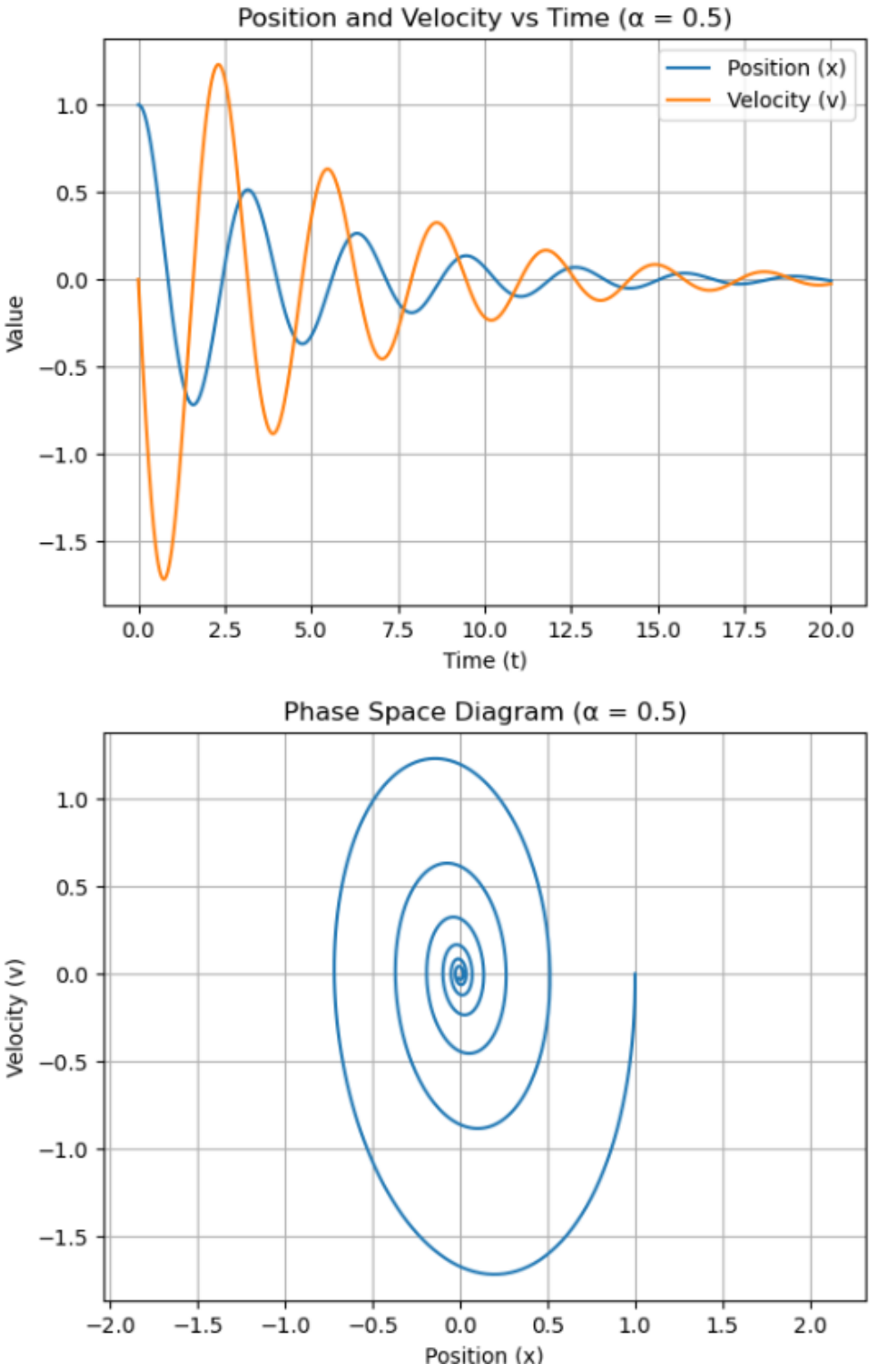

Figure 2.2.3 : Position and Velocity vs Time, Phase Space digram for α = 0.5

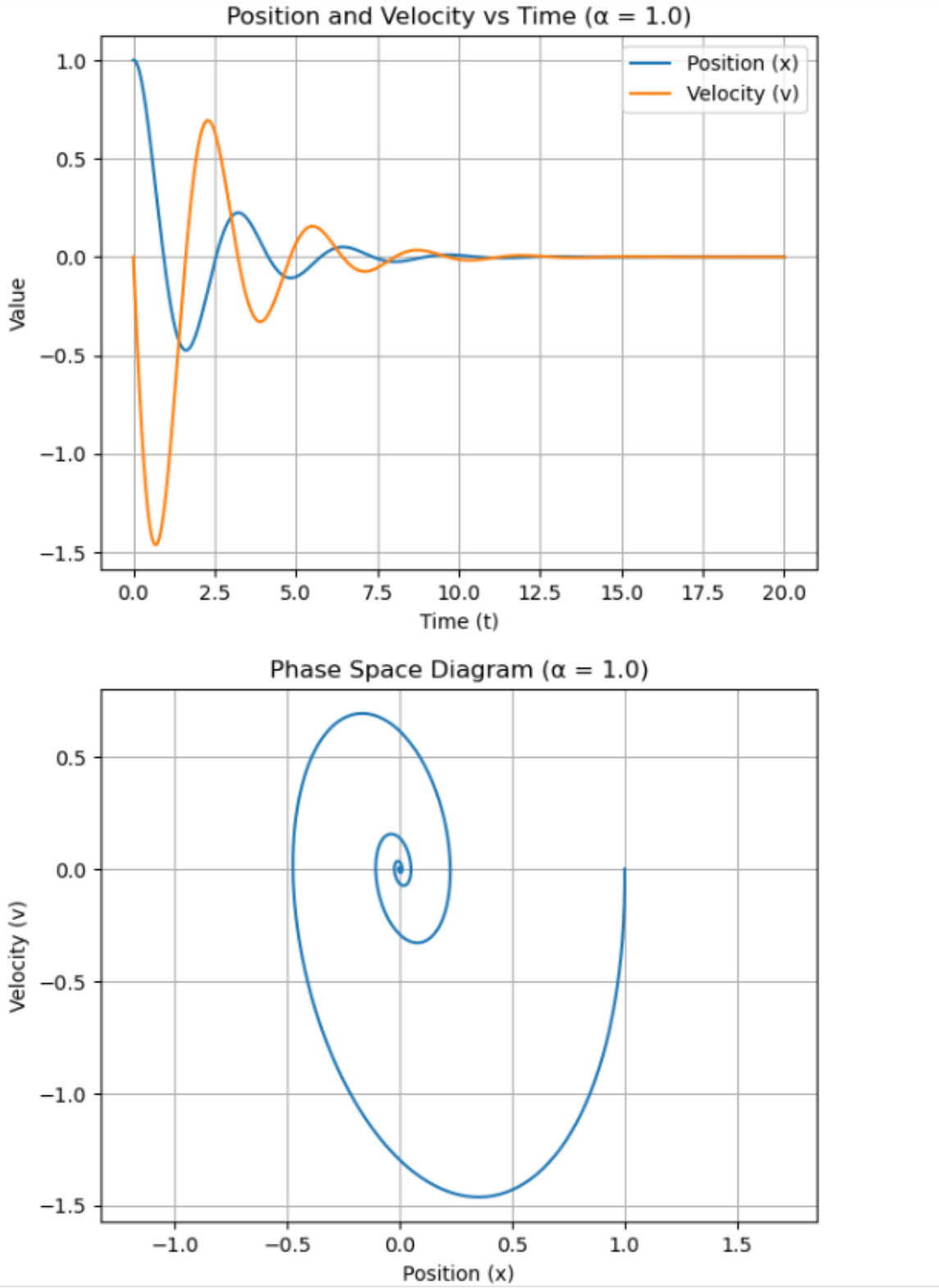

Figure 2.2.4 : Position and Velocity vs Time, Phase Space digram for α = 1.0

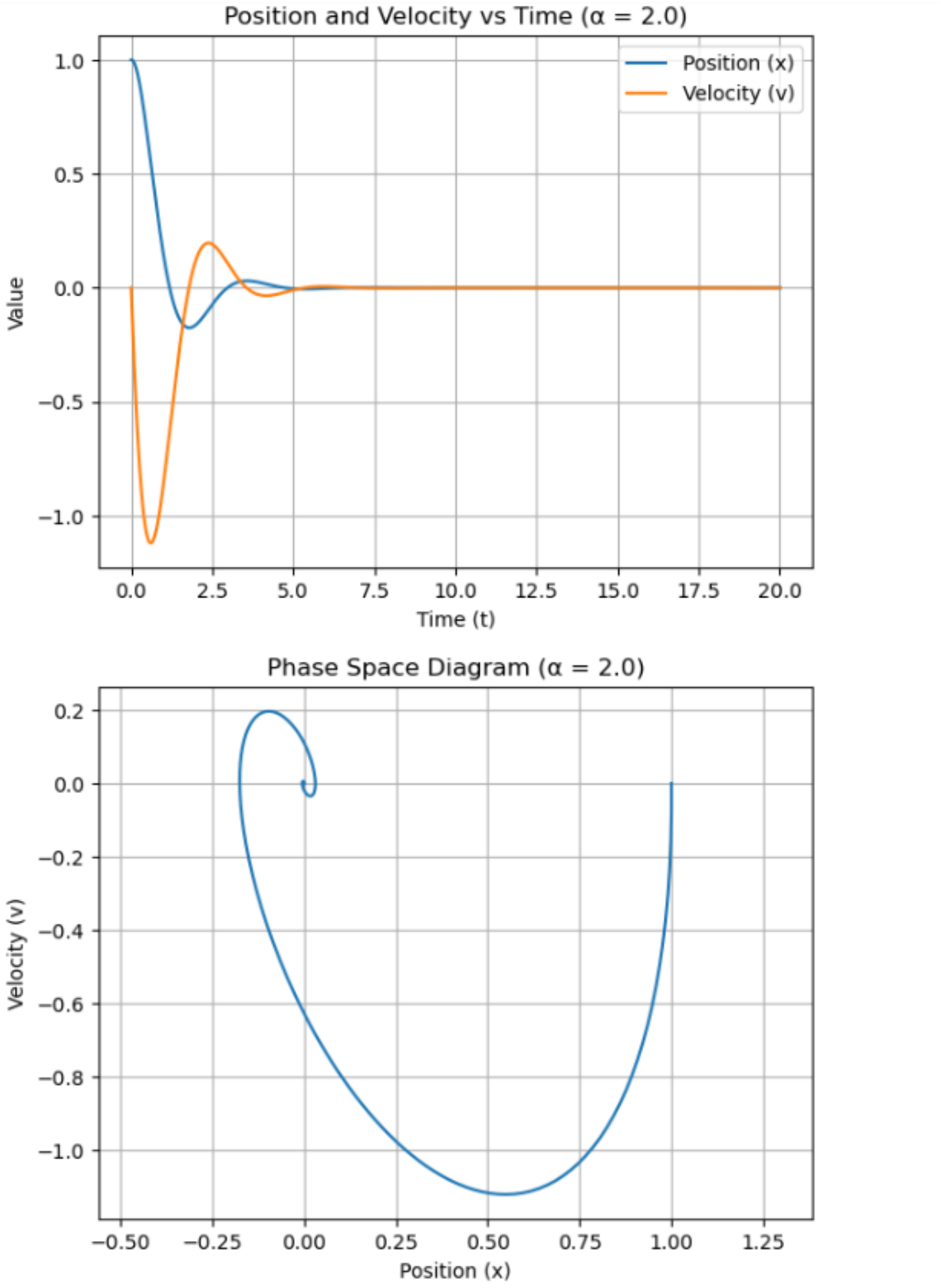

Figure 2.2.5 : Position and Velocity vs Time, Phase Space digram for α = 2.0

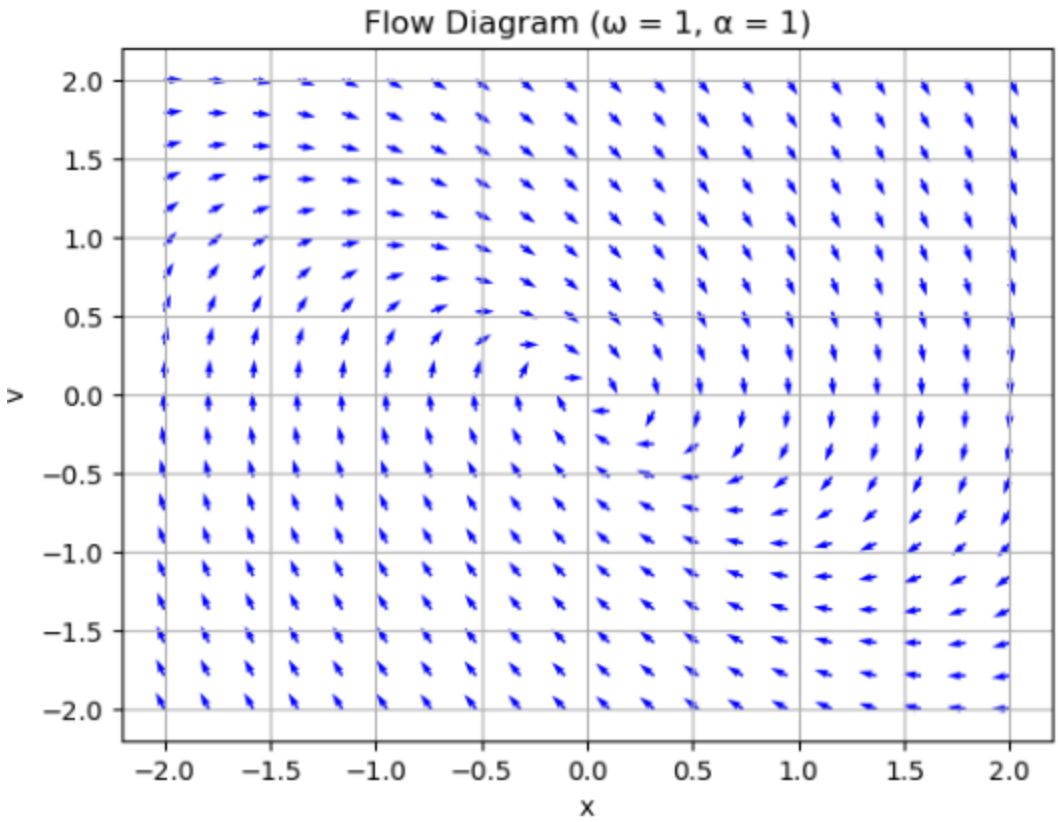

Figure 2.2.6 : Flow Diagram

2.2.5 Discussion

The simulation confirms that damping significantly affects the system's behavior. For low damping, the system exhibits oscillatory motion with gradually decreasing amplitude. As damping increases, oscillations reduce, and the system smoothly returns to equilibrium. The flow diagram and phase portraits clearly show convergence toward the origin, confirming it as a stable spiral point. This indicates that the system is underdamped and energy is steadily dissipated over time. The results align well with theoretical expectations, demonstrating how damping stabilizes dynamic systems by reducing motion and guiding them toward equilibrium, the Equilibrium point of $\alpha = 0.1$ *is (0, 0).*

2.3. Exercise 3 (Damped Harmonic Oscillator)

2.6.1 Objective

The main objective of this exercise is to analyze an unusual physical system: a harmonic oscillator with negative damping. This violates the typical assumption of energy dissipation and instead describes a system that gains energy. The goals of this activity is Formulate the model, find equilibrium, analyze stability, characterize the point and visualize the flow.

2.6.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

omega = 2.0
alpha_values = [-0.1, -0.5, -1.0, -2.0]

for alpha in alpha_values:
    t = np.linspace(0, 20, 1000)
    x0, v0 = 1.0, 0.0
    x = np.zeros_like(t)
    v = np.zeros_like(t)
    x[0], v[0] = x0, v0

    dt = t[1] - t[0]
    for i in range(1, len(t)):
        x[i] = x[i-1] + v[i-1] * dt
        v[i] = v[i-1] + (-omega**2 * x[i-1] - alpha * v[i-1]) * dt

    # Plot position and velocity vs time
    plt.figure()
    plt.plot(t, x, label='Position (x)')
    plt.plot(t, v, label='Velocity (v)')
    plt.xlabel('Time (t)')
    plt.ylabel('Value')
    plt.title(f'Position and Velocity vs Time (α = {alpha})')
    plt.legend()
    plt.grid()
    plt.savefig(f'position_velocity_alpha_{alpha}.png')

    # Phase space diagram
    plt.figure()
    plt.plot(x, v)
    plt.xlabel('Position (x)')
    plt.ylabel('Velocity (v)')
    plt.title(f'Phase Space Diagram (α = {alpha})')
    plt.grid()
    plt.axis('equal')
    plt.savefig(f'phase_space_alpha_{alpha}.png')
```

```python
# Flow diagram for alpha = -1, omega = 1
omega_flow = 1.0
alpha_flow = -1.0
x = np.linspace(-2, 2, 20)
y = np.linspace(-2, 2, 20)
X, Y = np.meshgrid(x, y)
dX = Y
dY = -omega_flow**2 * X - alpha_flow * Y
M = np.hypot(dX, dY)
dX = dX / M
dY = dY / M

plt.figure()
plt.quiver(X, Y, dX, dY, scale=50, color='b')
plt.xlabel('x')
plt.ylabel('v')
plt.title(f'Flow Diagram (ω = {omega_flow}, α = {alpha_flow})')
plt.grid()
plt.savefig('flow_diagram_neg_alpha.png')

x_eq, v_eq = 0, 0
print(f"Equilibrium point: ({x_eq}, {v_eq})")
A = np.array([[0, 1], [-omega_flow**2, -alpha_flow]])
eigenvalues = np.linalg.eigvals(A)
print(f"Eigenvalues: {eigenvalues}")
```

```
Equilibrium point: (0, 0)
Eigenvalues: [0.5+0.8660254j 0.5-0.8660254j]
```

Figure 2.3.1 : Code for Plot position and velocity, Plot the phase space diagram and plot the flow map for damped harmonic oscillator

2.6.3 How it works

The Python code simulates a damped pendulum using NumPy and Matplotlib, modeling the equation $\ddot{x} = -\omega^2 x - \alpha \dot{x}$ two first-order differential equations: $dx/dt = v$ and $dv/dt = -\omega^2 x - \alpha v$, where $\omega = 2.0$ is the natural frequency and α is the damping coefficient (varied as -0.1, -0.5, -1.0, -2.0).For each α, it creates a time array from 0 to 20 seconds with 1000 points, initializing position x and velocity v with starting values (1.0, 0.0). The Euler method numerically solves these equations iteratively, updating x and v based on the differential equations with a small-time step dt.

2.6.4 Results

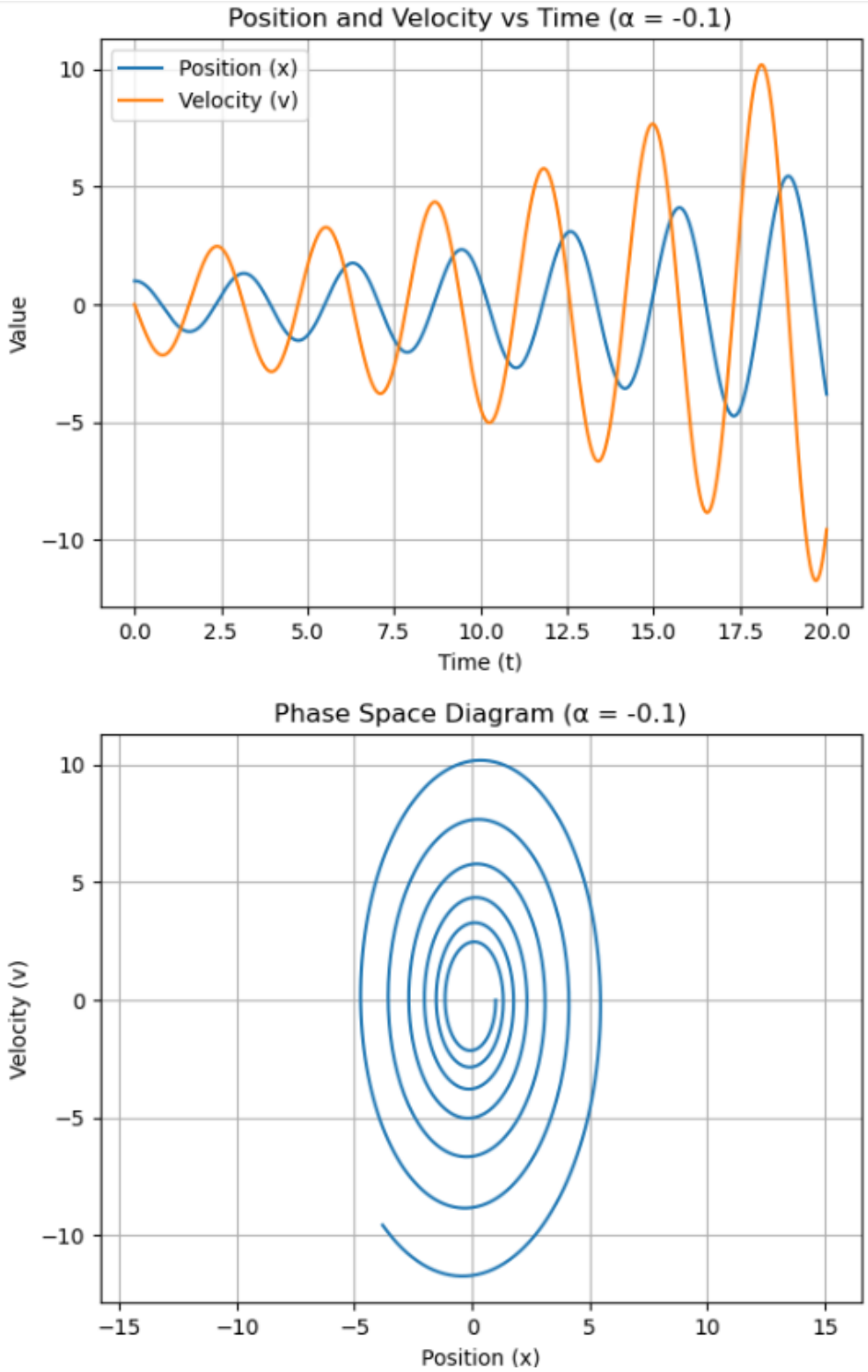

Figure 2.3.2 : Position and Velocity vs Time, Phase Space digram for α = -0.1

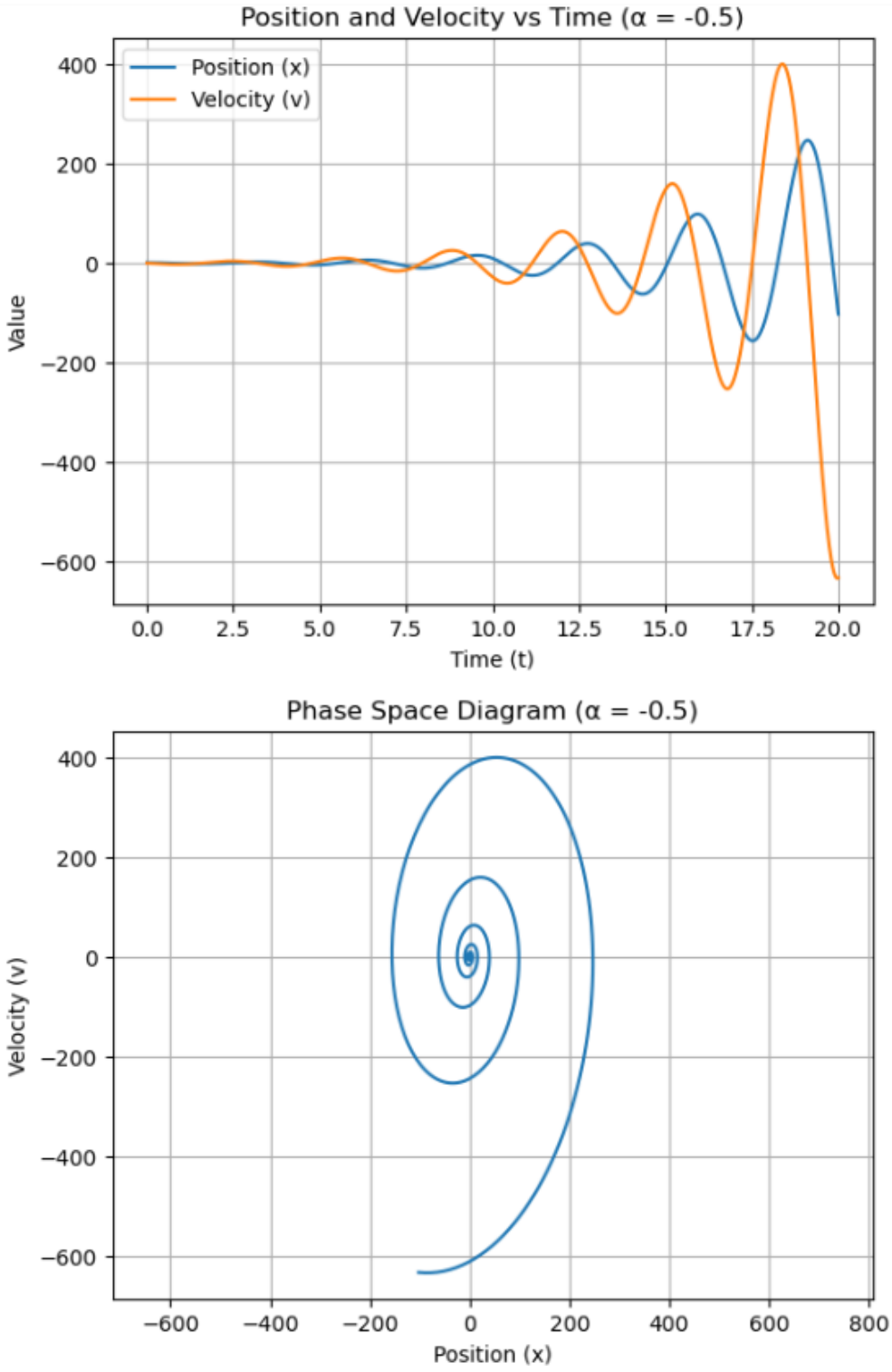

Figure 2.3.3 : Position and Velocity vs Time, Phase Space digram for α = -0.5

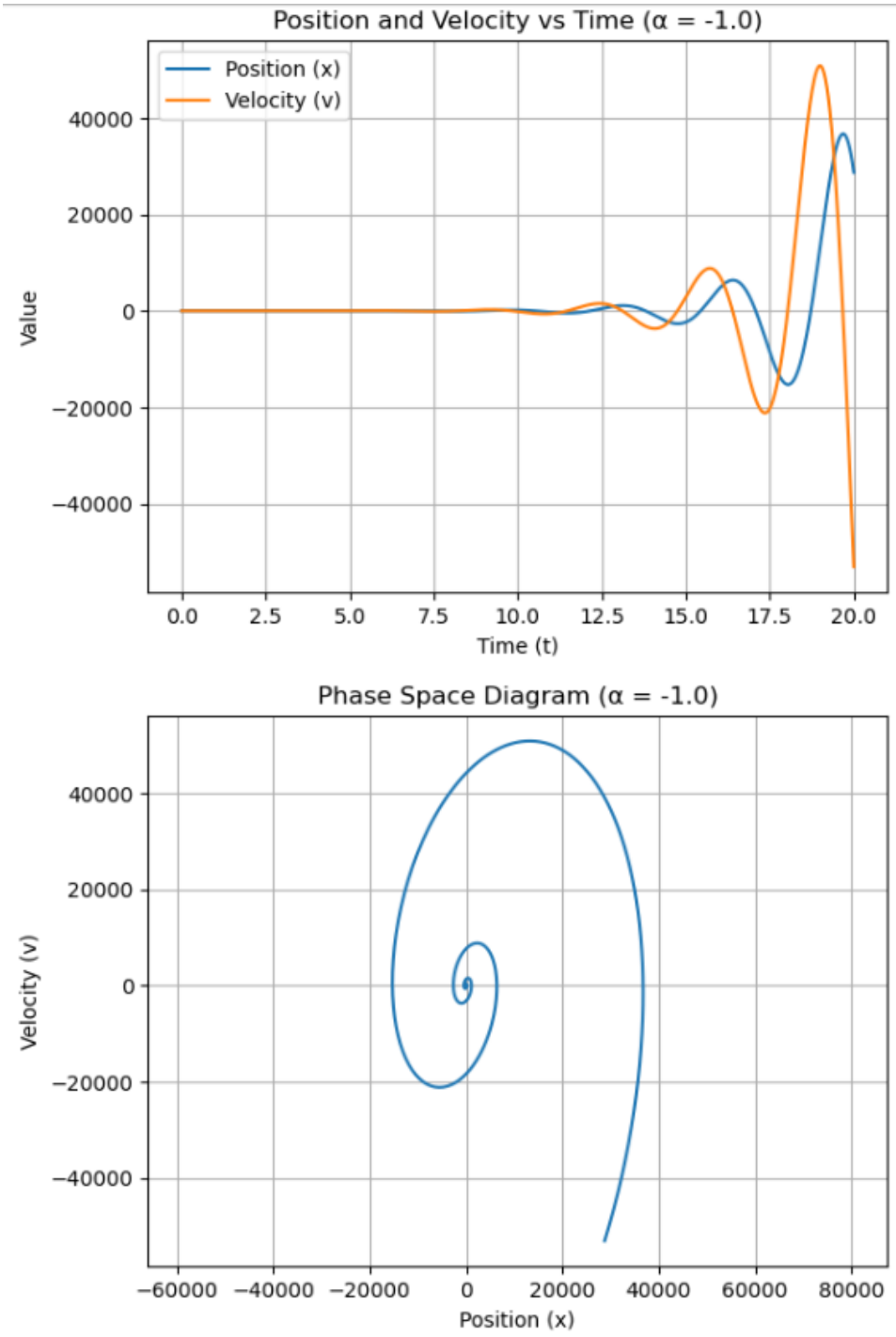

Figure 2.3.4 : Position and Velocity vs Time, Phase Space digram for α = -1.0

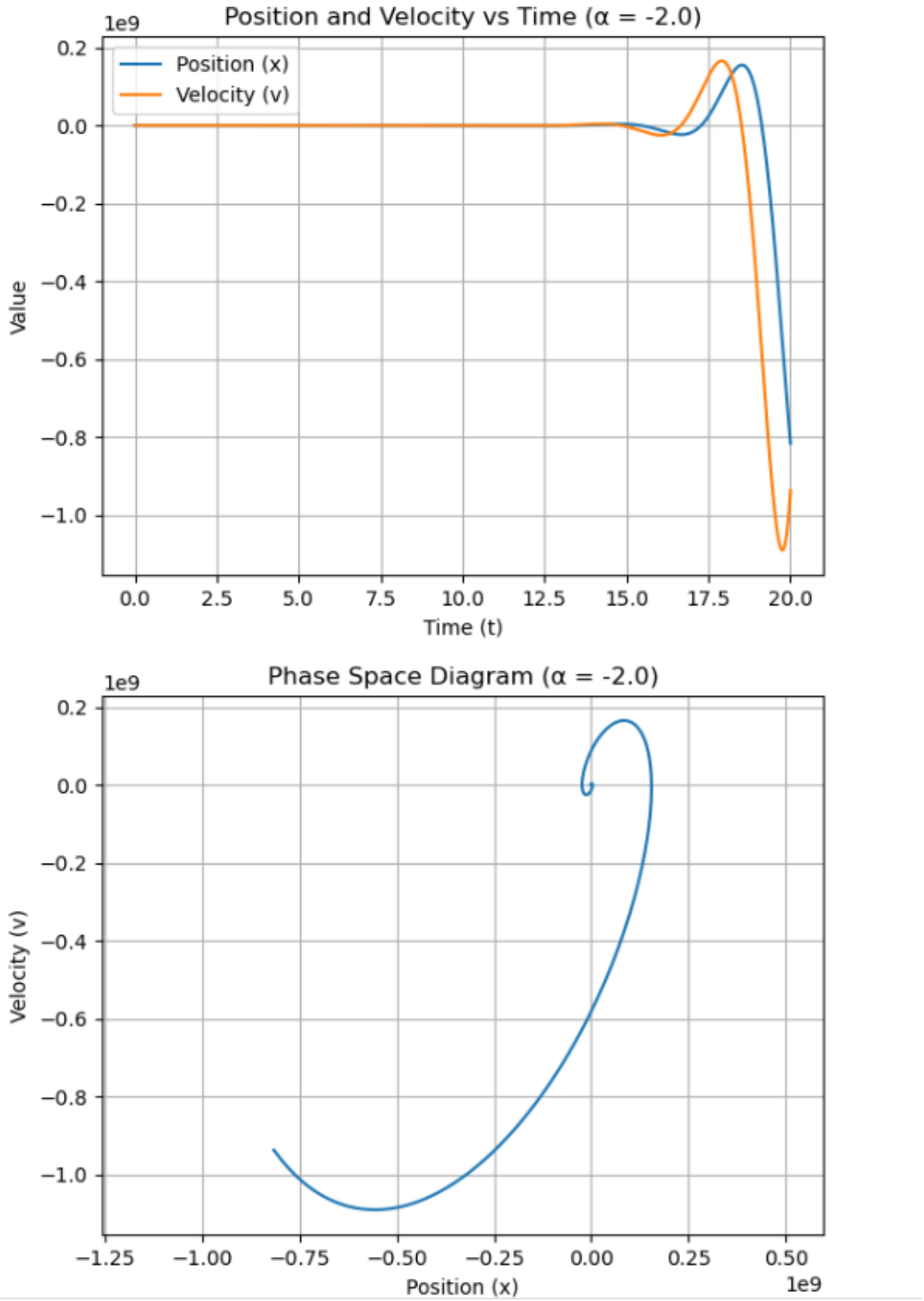

Figure 2.3.5 : Position and Velocity vs Time, Phase Space digram for α = -2.0

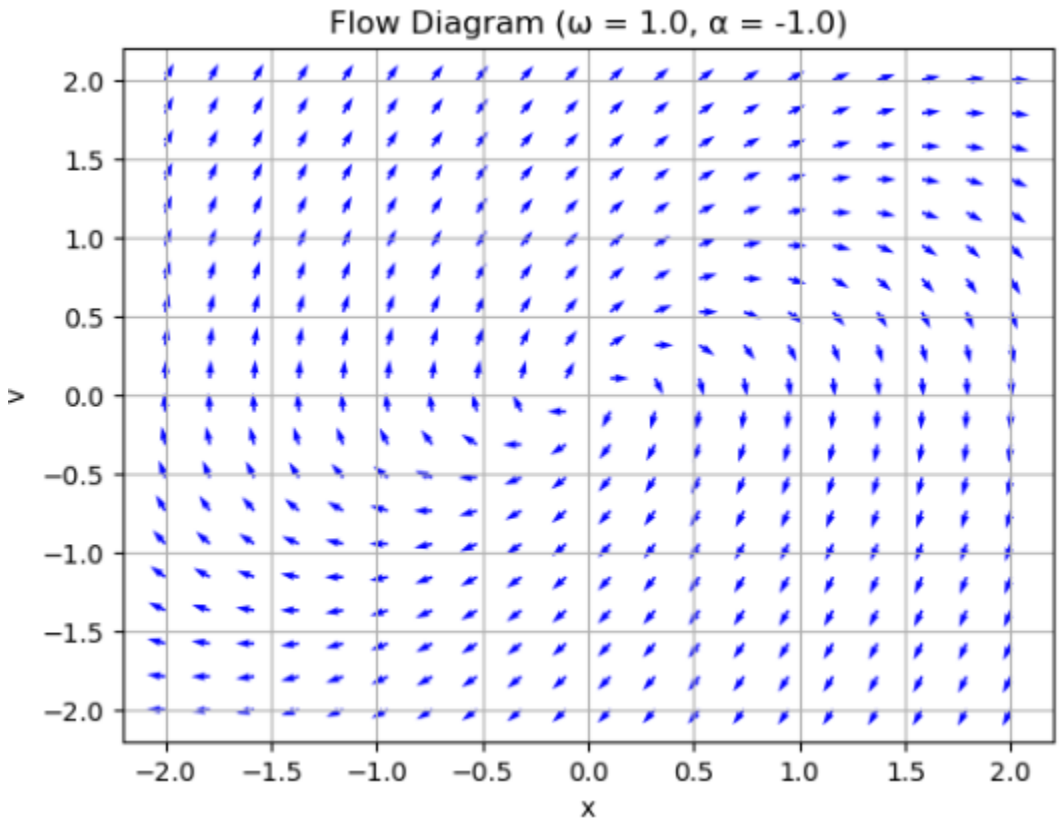

Figure 2.3.5 : Flow diagrame

2.6.5 Discussion

The Python simulation of a negatively damped harmonic oscillator provides a clear and consistent result: the system is fundamentally unstable. All simulations, regardless of the initial negative damping value, show that the oscillator's amplitude grows exponentially over time. This is visually confirmed by the time-series plots, which exhibit increasingly large oscillations, and the phase space diagrams, where trajectories spiral outward from the origin.

The mathematical analysis further validates these observations. The system's equilibrium point at (0,0) is shown to be an unstable spiral point, as evidenced by the eigenvalues of the Jacobian matrix having a positive real part. The negative damping term fundamentally changes the system's dynamics, turning an energy-dissipating system into an energy-creating one, and transforming a stable equilibrium into an unstable one.

2.4. Exercise 4 (Forced pendulum with damping)

2.4.1 Objective

The main object of this exercise is to analyze the behavior of a forced damped pendulum which is a classic example of a driven oscillatory system. Another object of this exercise is converting the second – order differential equation into a system of two first – order differential equation. Also, this exercise includes Numerical simulation, Phase space analysis and plot the system's trajectories in phase space for a specific set of parameters.

2.4.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

alpha = 0.5
A = 5.0
omega = 2.0

t_max = 100
num_points = 5000
t = np.linspace(0, t_max, num_points)
dt = t[1] - t[0]

initial_conditions = [
    (1.0, 0.0),
    (-2.0, 1.0),
    (0.0, -3.0),
    (3.0, 3.0)
]

plt.figure(figsize=(10, 8))

for i, (x0, v0) in enumerate(initial_conditions):
    x = np.zeros_like(t)
    v = np.zeros_like(t)
    x[0], v[0] = x0, v0

    for j in range(1, len(t)):
        forcing_term = A * np.cos(omega * t[j-1])

        x[j] = x[j-1] + v[j-1] * dt
        v[j] = v[j-1] + (-omega**2 * x[j-1] - alpha * v[j-1] + forcing_term) * dt

    plt.plot(x, v, label=f'Initial Cond: ({x0}, {v0})')

plt.xlabel('Position (x)')
plt.ylabel('Velocity (v)')
plt.title(f'Phase Space Diagrams (α = {alpha}, A = {A})')
plt.legend()
plt.grid()
```

Figure 2.4.1 : Code for phase space diagram

2.4.3 How it Works

This code simulates a forced, damped pendulum by numerically solving its differential equations. First, the second-order equation $\ddot{x} = -\omega^2 x - \alpha \dot{x} + A\cos(\omega t)$ is converted into a system of two first-order equations: $\dot{x} = v$ and $\dot{v} = -\omega^2 x - \alpha v + A\cos(\omega t)$.The simulation then uses Euler's method, a numerical integration technique, to step through time. For each small-time increment, it calculates the new position and velocity based on the previous values and the forces acting on the system, including the spring force, damping force, and the external periodic forcing term ($A\cos(\omega t)$). To find a long-term behavior pattern, the code runs the simulation for several different initial conditions. All these trajectories are plotted together on a single-phase space diagram (velocity vs. position). The key result is that, after an initial transient period, all trajectories converge to a single closed loop, which is the system's limit cycle attractor.

2.4.4 Results

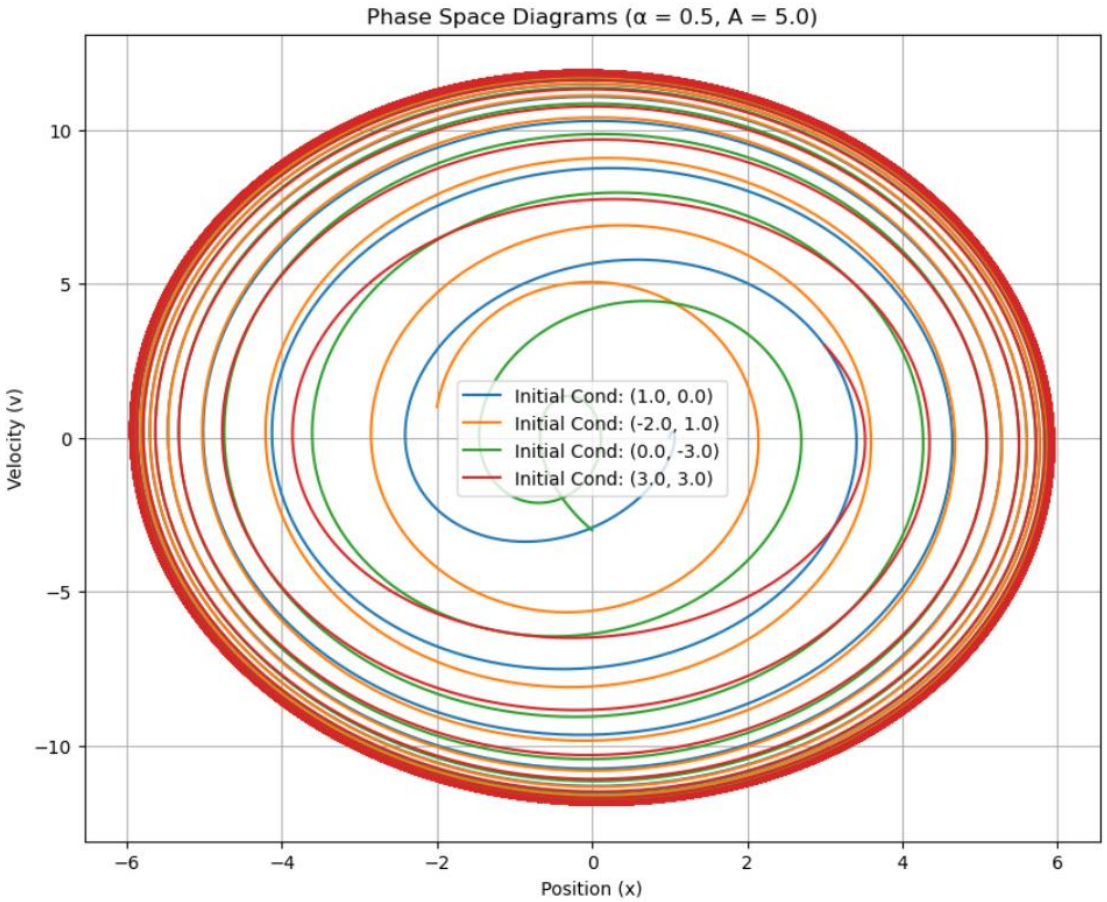

Figure 2.4.2 : Phase space diagrams

2.4.5 Discussion

The Python code effectively demonstrates the behavior of a forced, damped harmonic oscillator. The simulation, which solves the system's differential equations using Euler's method, reveals that the long-term behavior is independent of the initial conditions. Based on the physics of a forced, damped harmonic oscillator and the results of the Python code, the attractor is a limit cycle.

By plotting trajectories from multiple starting points on a single phase space diagram, the code clearly shows that all paths eventually converge onto a stable, repeating orbit. This stable, closed loop in phase space is a limit cycle attractor. It represents the steady-state motion of the system, where the energy dissipated by damping is perfectly balanced by the energy injected by the external forcing term.

The conclusion is that for this forced, damped system, the transient behavior is influenced by initial conditions, but the system's eventual fate is to settle into a predictable and stable periodic motion, confirming the existence of a limit cycle attractor

2.5. Exercise 5 (The motion of a pendulum)

2.5.1 Objective

The main objective of this exercise is to analyze the long- term behavior of a forced, damped pendulum system. Mathematical formulation, Convert the given second – order differential equation into a system of two first – order equation. Solve these equations using numerical method for specified parameters and plot the trajectories in phase space to visualize the system's dynamics. The final objective is to observe if the system converges to a stable attractor.

2.5.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

g = 9.81
L = 1.0

def pendulum(t, z):
    x, y = z
    dxdt = y
    dydt = -(g/L) * np.sin(x)
    return [dxdt, dydt]

x = np.linspace(-4 * np.pi, 4 * np.pi, 40)
y = np.linspace(-10, 10, 40)
X, Y = np.meshgrid(x, y)
U = Y
V = -(g / L) * np.sin(X)

plt.figure(figsize=(10, 6))
plt.streamplot(X, Y, U, V, density=1.2, arrowsize=1)
plt.title('Pendulum Phase Portrait (Flow Map)')
plt.xlabel('Angle (x)')
plt.ylabel('Angular velocity (y)')
plt.grid()
plt.show()

x_zoom = np.linspace(2, 4, 30)
y_zoom = np.linspace(-2, 2, 30)
Xz, Yz = np.meshgrid(x_zoom, y_zoom)
Uz = Yz
Vz = -(g / L) * np.sin(Xz)
```

```
plt.figure(figsize=(8, 5))
plt.streamplot(Xz, Yz, Uz, Vz, density=1.2, arrowsize=1)
plt.title('Zoomed Flow Diagram near Saddle Node')
plt.xlabel('Angle (x)')
plt.ylabel('Angular velocity (y)')
plt.grid()
plt.show()

def simulate_and_plot(theta0, y0):
    t_span = (0, 10)
    t_eval = np.linspace(*t_span, 1000)
    sol = solve_ivp(pendulum, t_span, [theta0, y0], t_eval=t_eval)
    plt.plot(sol.y[0], sol.y[1], label=f"x₀ = {np.round(theta0, 2)}, y₀ = {y0}")

initial_angles = [np.pi/6, np.pi/4, np.pi/3, np.pi/2, 3*np.pi/4, 5*np.pi/6, 7*np.pi/6]

plt.figure(figsize=(10, 6))
for angle in initial_angles:
    simulate_and_plot(angle, 0)
plt.title("Angular Velocity vs Angle (y vs x) for Various Initial Angles")
plt.xlabel("Angle (x)")
plt.ylabel("Angular velocity (y)")
plt.grid()
plt.legend()
plt.show()

# High velocity test
plt.figure(figsize=(8, 5))
simulate_and_plot(np.pi/6, 10)
plt.title("High Velocity Case: y₀ = 10")
plt.xlabel("Angle (x)")
plt.ylabel("Angular velocity (y)")
plt.grid()
plt.legend()
plt.show()
```

Figure 2.5.1: The code of plot flow map and other plots

2.5.3 How it Works

This Python code simulates and visualizes the motion of a nonlinear pendulum using numerical methods. It defines the pendulum's equations of motion as a system of first-order differential equations. The first part generates a flow map showing how the pendulum's angle and angular velocity evolve for various initial conditions. A zoomed-in flow map highlights the region near saddle nodes unstable equilibrium points. Next, the code uses `solve_ivp` from SciPy to numerically solve the differential equations for specific initial angles, assuming the pendulum is released from rest. It plots angular velocity versus angle for multiple initial positions, including a high-velocity case to observe different dynamic behaviors. These plots help visualize stable oscillations, rotation, and the influence of initial conditions on motion. Overall, the code provides a local view of pendulum dynamics.

2.5.4 Results

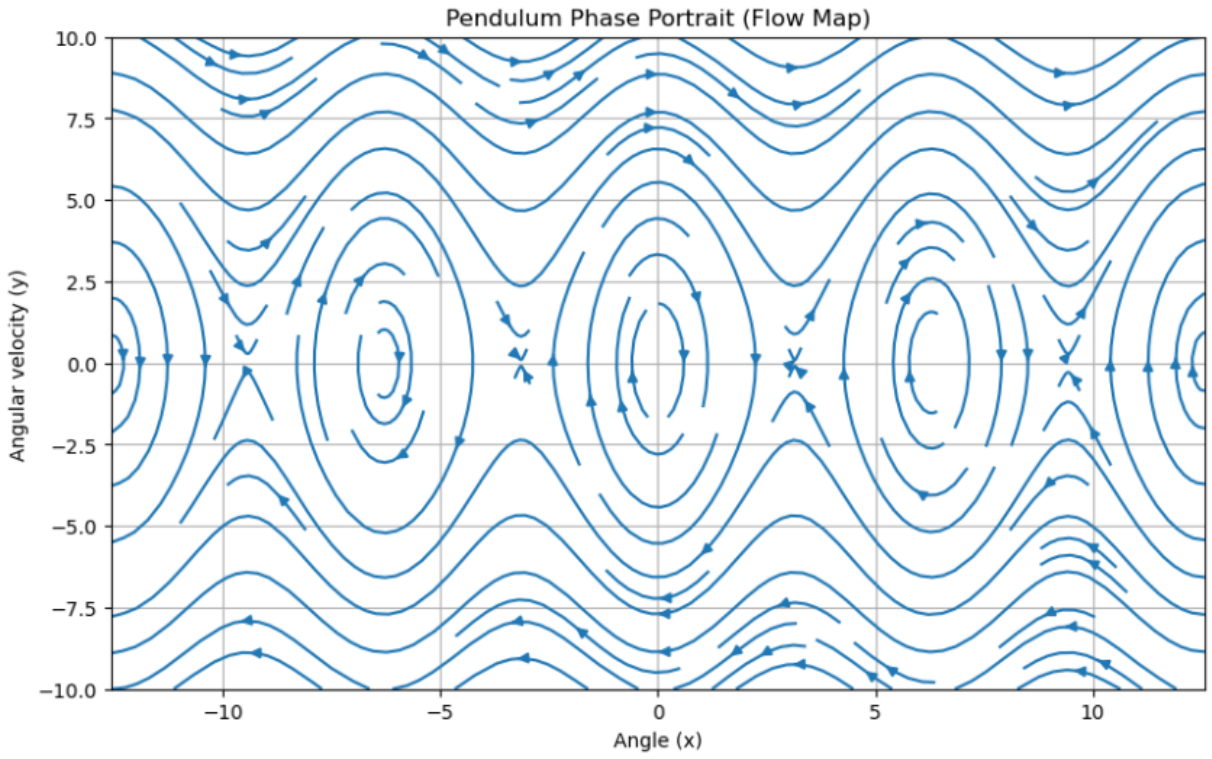

Figure 2.5.2: Flow map for Pendulum phase portrait

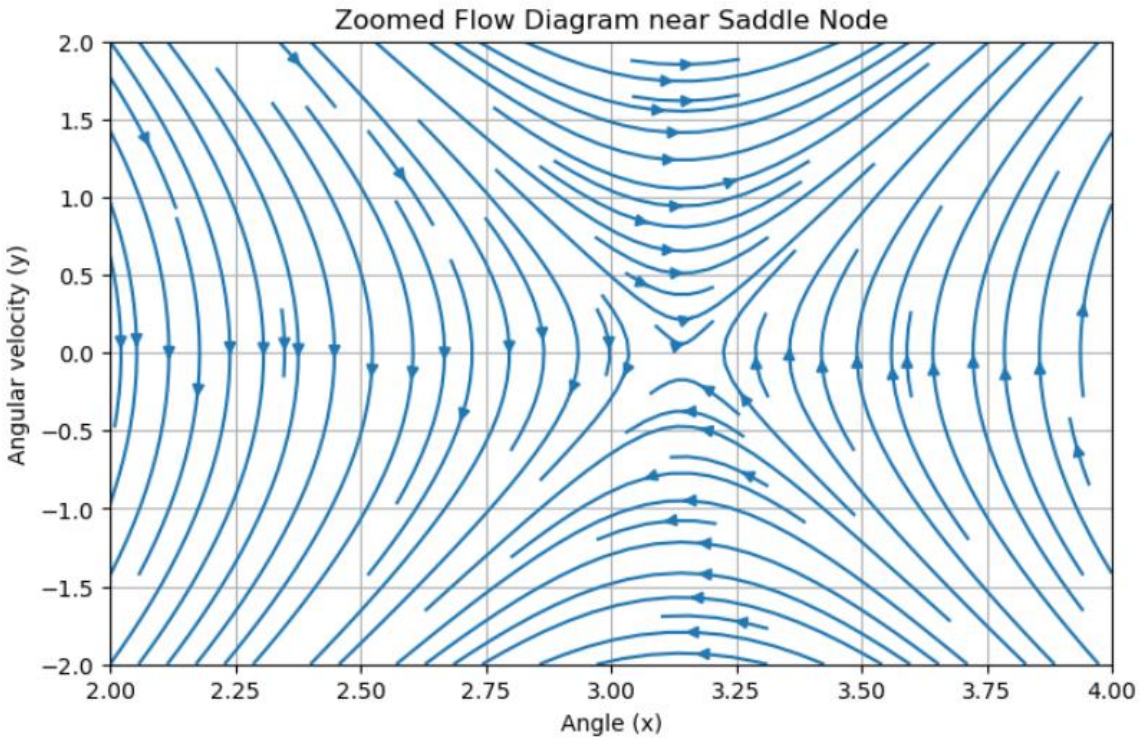

Figure 2.5.3: Flow Diagram near Saddle Node

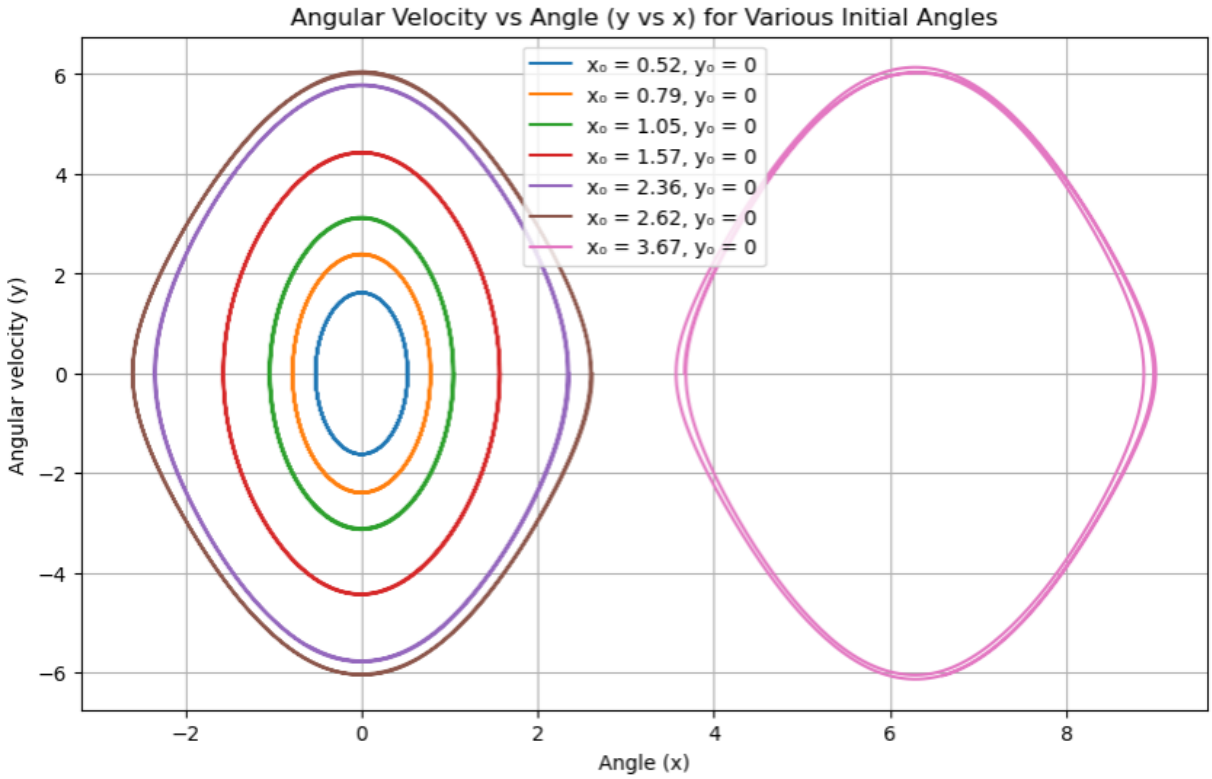

Figure 2.5.4: Angular velocity vs Angle for various initial angles

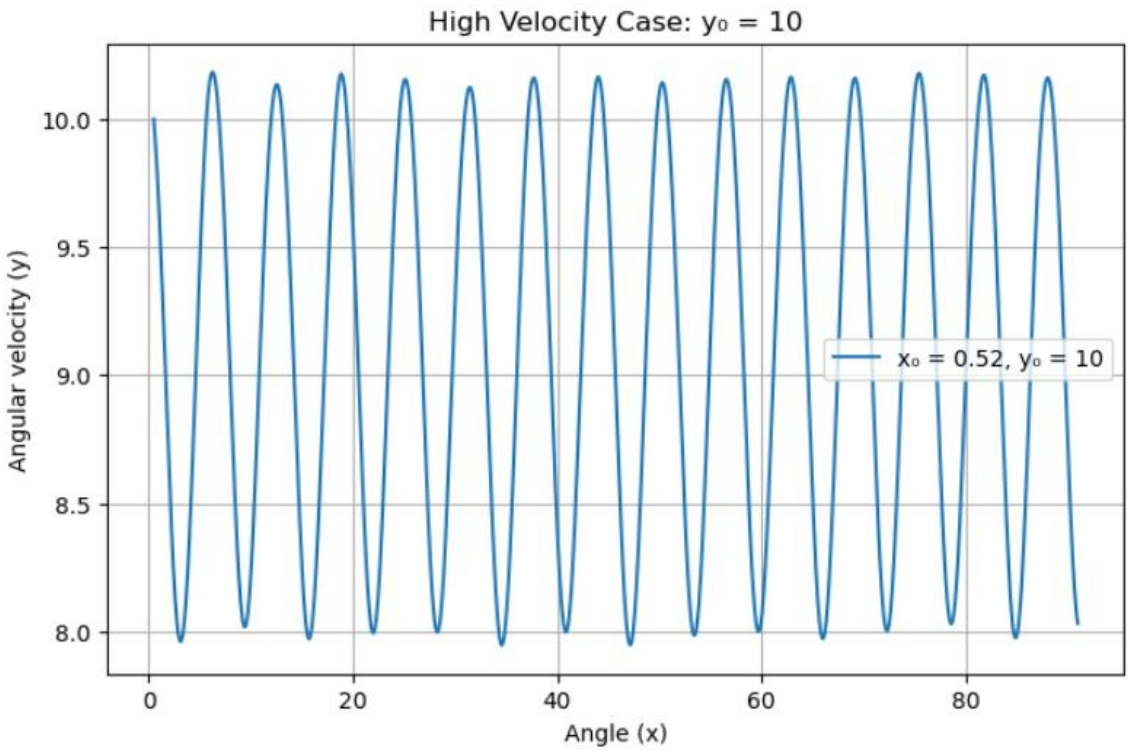

Figure 2.5.5: High velocity case

2.5.5 Discussion

The global flow map of the nonlinear pendulum clearly reveals the system's complex dynamics under varying initial conditions. For small angles and moderate angular velocities, the pendulum exhibits smooth, closed trajectories in phase space, indicating regular, periodic oscillations. Zooming into the region between x=2x = 2 and x=4x = 4, the flow shows a saddle node near x=πx = \pi, representing an unstable equilibrium where paths diverge and converge along different axes. When released from rest at angles less than π\pi, the pendulum displays symmetric, bounded oscillations, forming closed loops. For initial angles greater than π\pi, the motion remains bounded due to gravity's restoring force, but the phase paths shift because of the circular nature of angular motion. With high initial angular velocity, the pendulum transitions to continuous rotation, producing open, unbounded paths in phase space, signifying non-periodic dynamics.

Answers:

(a) Periodic motion occurs when the energy is low and the pendulum oscillates without rotating.

(b) Released from rest at angles less than π\pi, the motion is periodic and forms closed loops.

(c) From angles greater than π\pi, the system returns to oscillation, though phase paths shift.

(d) High initial velocity leads to rotational motion and open, non-repeating trajectories.

2.6. Exercise 6 (Van der Pol oscillator)

2.7.1 Objective

The object of this experiment is to explore the dynamics of the Van der Pol oscillator, a well-known nonlinear system that exhibits self-sustained oscillations. This system is described by a second-order differential equation featuring a nonlinear damping term. This damping behaves uniquely enhancing small oscillations while damping larger ones which results in a stable, repeating pattern called a limit cycle. The experiment involves numerically solving the Van der Pol equation using selected initial conditions, plotting its trajectory in phase space, and creating a flow diagram to study the system's overall behavior. By examining how various trajectories across the phase space converge toward a consistent closed loop, the experiment illustrates the existence and stability of the limit cycle characteristic of the Van der Pol oscillator.

2.6.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def van_der_pol(t, y, mu):
    x, dx = y
    return [dx, mu * (1 - x**2) * dx - x]

mu = 1.0
y0 = [1.0, 0.1]
t_span = (0, 20)
t_eval = np.linspace(*t_span, 1000)

solution = solve_ivp(van_der_pol, t_span, y0, args=(mu,), t_eval=t_eval)

x = solution.y[0]
y = solution.y[1]

# Plot phase portrait
plt.figure(figsize=(8, 6))
plt.plot(x, y, color='blue', lw=1.5, label="Trajectory")
plt.title("Van der Pol Oscillator Phase Portrait")
plt.xlabel("x (Position)")
plt.ylabel("dx/dt (Velocity)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.xlim(-3, 3)
plt.ylim(-5, 5)
plt.show()
```

Figure 2.6.1: The code for solve Van der pol oscillator and plot its trajectory

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def van_der_pol(t, y, mu):
    x, dx = y
    return [dx, mu * (1 - x**2) * dx - x]

mu = 1.0

x_vals = np.arange(-4, 4.1, 0.5)
dx_vals = np.arange(-6, 6.1, 0.5)
X, DX = np.meshgrid(x_vals, dx_vals)

DXU = DX
DYU = mu * (1 - X**2) * DX - X

magnitude = np.sqrt(DXU**2 + DYU**2) + 1e-6
DXU_norm = DXU / magnitude
DYU_norm = DYU / magnitude

y0 = [1.0, 1.0]
t_span = (0, 20)
t_eval = np.linspace(*t_span, 1000)

solution = solve_ivp(van_der_pol, t_span, y0, args=(mu,), t_eval=t_eval)
x = solution.y[0]
dx = solution.y[1]

plt.figure(figsize=(8, 6))
plt.quiver(X, DX, DXU_norm, DYU_norm, scale=20, width=0.005, color='gray', alpha=0.6, label='Flow Field')
plt.plot(x, dx, color='blue', lw=2, label='Trajectory')
plt.title("Flow Map and Trajectory for Van der Pol Oscillator")
plt.xlabel("x (Position)")
plt.ylabel("dx/dt (Velocity)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

Figure 2.6.2: The code for the flow diagram of the Van der Pol oscillator and above trajectory on.

2.6.3 How it works

Both codes simulate and visualize the behavior of the Van der Pol oscillator, a nonlinear system with self-sustained oscillations. The first code numerically solves the Van der Pol differential equation using `solve_ivp` for a given initial condition, then plots the phase trajectory (position vs. velocity) to show how the system evolves over time. The second code extends this by adding a flow map, which shows a vector field representing the system's direction and speed of motion at various points in phase space. It computes and normalizes these vectors across a grid, and overlays the trajectory from the first code. This helps visualize both the global dynamics and specific behavior of the system, revealing the presence of a stable limit cycle - a closed trajectory that attracts nearby paths. Together, these codes demonstrate how the Van der Pol oscillator behaves for different conditions and illustrate its non-linear, energy-regulating nature.

2.6.4 Results

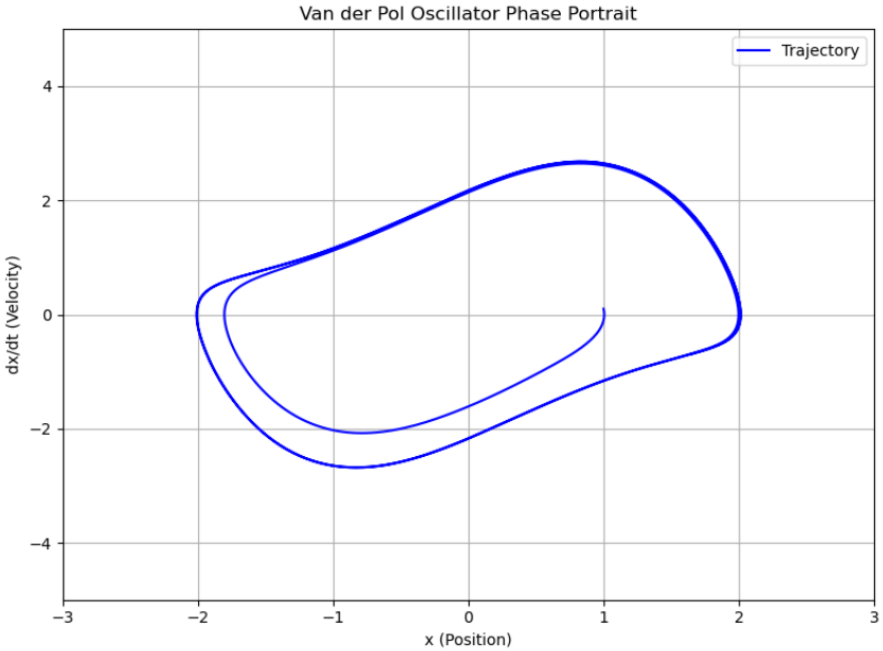

Figure 2.6.3: A closed curve of equation trajectory

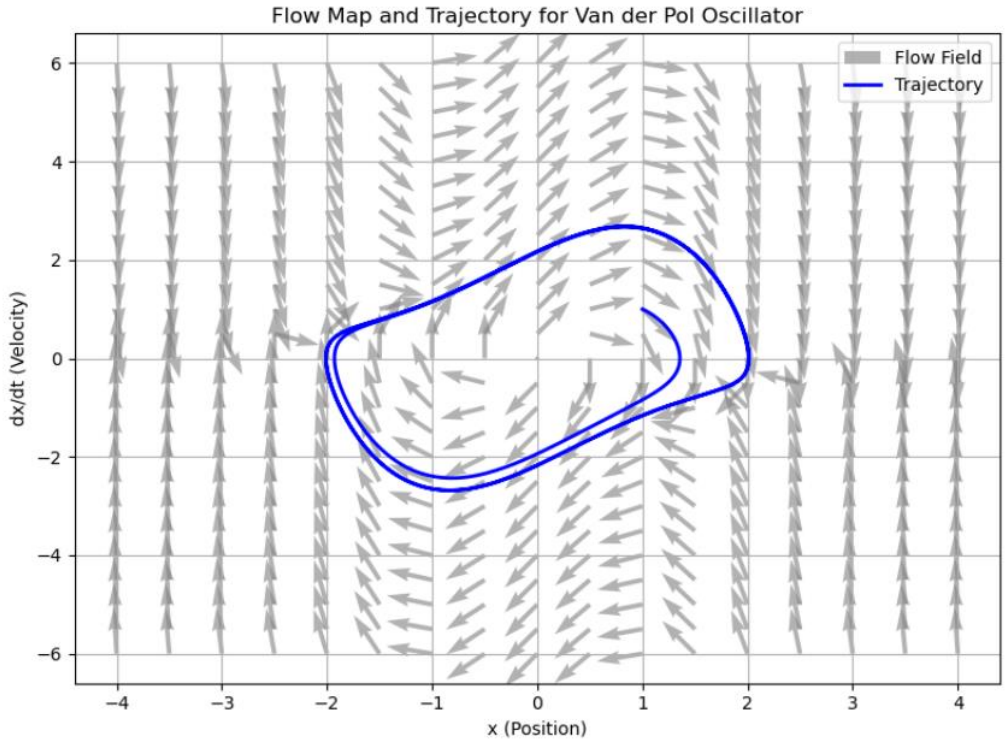

Figure 2.6.4: The flow map of the Van der pol oscillator and the same graph where above plotted

2.6.5 Discussion

The trajectory plotted from the initial condition (1.0,1.0) in the phase space forms a closed curve, clearly indicating a limit cycle. Unlike undamped harmonic oscillators that can have trajectories depending heavily on initial conditions, the Van der Pol oscillator exhibits behavior where nearby trajectories converge onto a single, stable loop. The flow map further confirms this: regardless of whether a point starts inside or outside the loop, the direction of the vector field points toward the limit cycle. This means the system naturally corrects its energy over time, increasing oscillation if it's too small and damping it if it's too large, until it stabilizes at a consistent amplitude and frequency. This self-stabilizing behavior is a defining characteristic of the Van der Pol oscillator and highlights the system's ability to reach a dynamic equilibrium. The flow map shows a global view of the system's dynamics, emphasizing the nonlinear damping effect that drives the system toward stable oscillation. This is especially significant in physical systems like electrical circuits, heartbeats, or biological rhythms, where sustained, stable oscillations are essential. The experiment successfully demonstrates the existence and stability of the limit cycle and illustrates the powerful use of phase portraits and flow maps in analyzing nonlinear systems.

2.7. Exercise 7 (Lorentz System)

2.7.1 Objective

The main object of this activity to investigate the dynamics of the Lorenz system, a series of nonlinear differential equations originally developed for atmospheric convection models. The primary objective is to numerically solve these equations for various parameter values to understand the complex, chaotic behavior they are known to produce. We will visualize the system's trajectories in both two and three-dimensional phase space, analyze its sensitivity to initial conditions, and plot the evolution of its variables over time. This research seeks to provide insight into the emergence of chaos, the nature of attractors, and the fundamental unpredictability found in specific nonlinear dynamical systems.

2.7.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

def lorenz(state, t, sigma, rho, beta):
    x, y, z = state
    dx_dt = sigma * (y - x)
    dy_dt = x * (rho - z) - y
    dz_dt = x * y - beta * z
    return [dx_dt, dy_dt, dz_dt]

t = np.linspace(0, 50, 10000)

params = [
    (14, 10, 8/3),
    (15, 10, 8/3),
    (28, 10, 8/3)
]

# Initial conditions
initial_conditions = [
    [1.0, 1.0, 1.0],
    [1.01, 1.0, 1.0]
]
```

```
for rho, sigma, beta in params:
    for ic in initial_conditions:

        solution = odeint(lorenz, ic, t, args=(sigma, rho, beta))
        x, y, z = solution.T

        plt.figure(figsize=(15, 5))

        plt.subplot(131)
        plt.plot(x, y)
        plt.title(f'x vs y (ρ={rho}, σ={sigma}, β={beta})')
        plt.xlabel('x')
        plt.ylabel('y')

        plt.subplot(132)
        plt.plot(y, z)
        plt.title(f'y vs z (ρ={rho}, σ={sigma}, β={beta})')
        plt.xlabel('y')
        plt.ylabel('z')

        plt.subplot(133)
        plt.plot(x, z)
        plt.title(f'x vs z (ρ={rho}, σ={sigma}, β={beta})')
        plt.xlabel('x')
        plt.ylabel('z')
        plt.tight_layout()
        plt.show()

        fig = plt.figure(figsize=(10, 8))
        ax = fig.add_subplot(111, projection='3d')
        ax.plot(x, y, z, lw=0.5)
        ax.set_title(f'3D Phase Space (ρ={rho}, σ={sigma}, β={beta})')
        ax.set_xlabel('x')
        ax.set_ylabel('y')
        ax.set_zlabel('z')
        plt.show()
```

```
        plt.figure(figsize=(15, 5))

        plt.subplot(131)
        plt.plot(t, x)
        plt.title(f'x vs t (ρ={rho}, σ={sigma}, β={beta})')
        plt.xlabel('t')
        plt.ylabel('x')

        plt.subplot(132)
        plt.plot(t, y)
        plt.title(f'y vs t (ρ={rho}, σ={sigma}, β={beta})')
        plt.xlabel('t')
        plt.ylabel('y')

        plt.subplot(133)
        plt.plot(t, z)
        plt.title(f'z vs t (ρ={rho}, σ={sigma}, β={beta})')
        plt.xlabel('t')
        plt.ylabel('z')
        plt.tight_layout()
        plt.show()
```

Figure 2.7.1: The code for Lorentz system to plot the phase space diagrams

2.7.3 How it works

The script simulates the Lorenz system, a set of nonlinear differential equations that model fluid convection and exhibit chaotic behavior under certain conditions. It uses scipy. integrate. odeint to numerically solve these equations and matplotlib to visualize the results as phase space diagrams and time series plots. The code explores different parameter sets and initial conditions to demonstrate chaotic and non-chaotic behavior.

2.7.4 Results

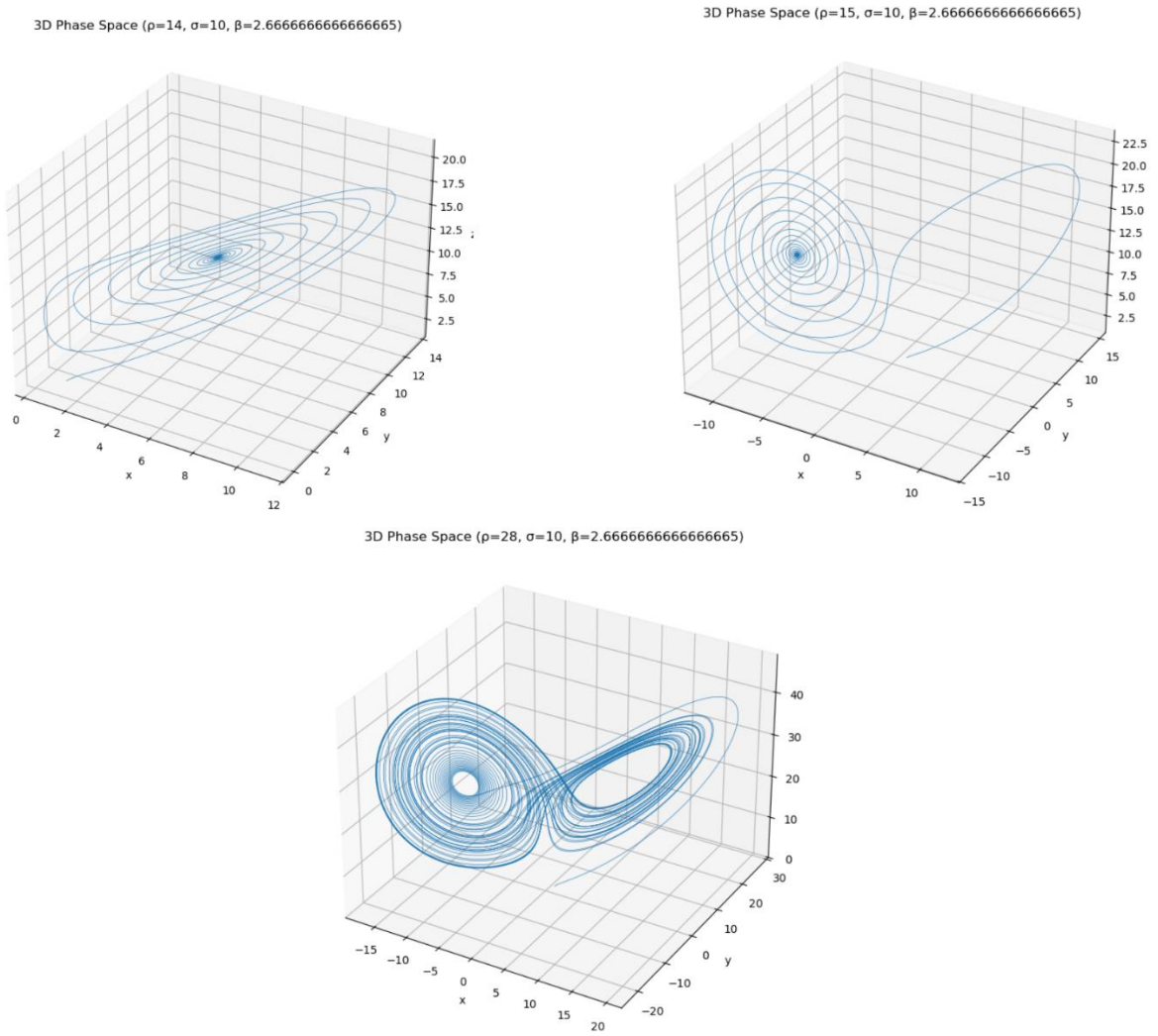

Figure 2.7.2: Lorentz system graph of phase space diagrams(3d)

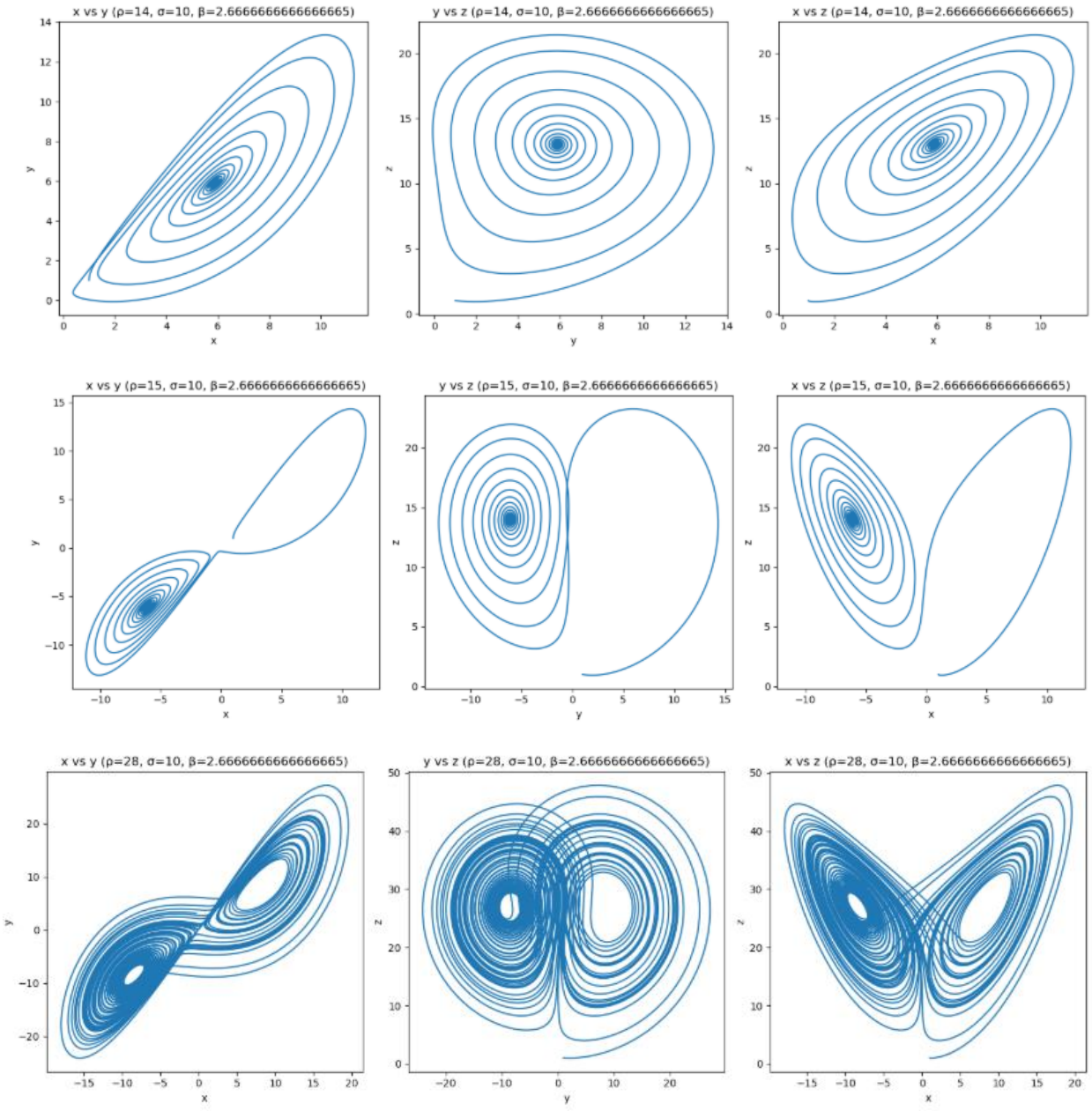

Figure 2.7.3: Lorentz system graph of phase space diagrams 2d. Here x vs y and y vs z graphs.

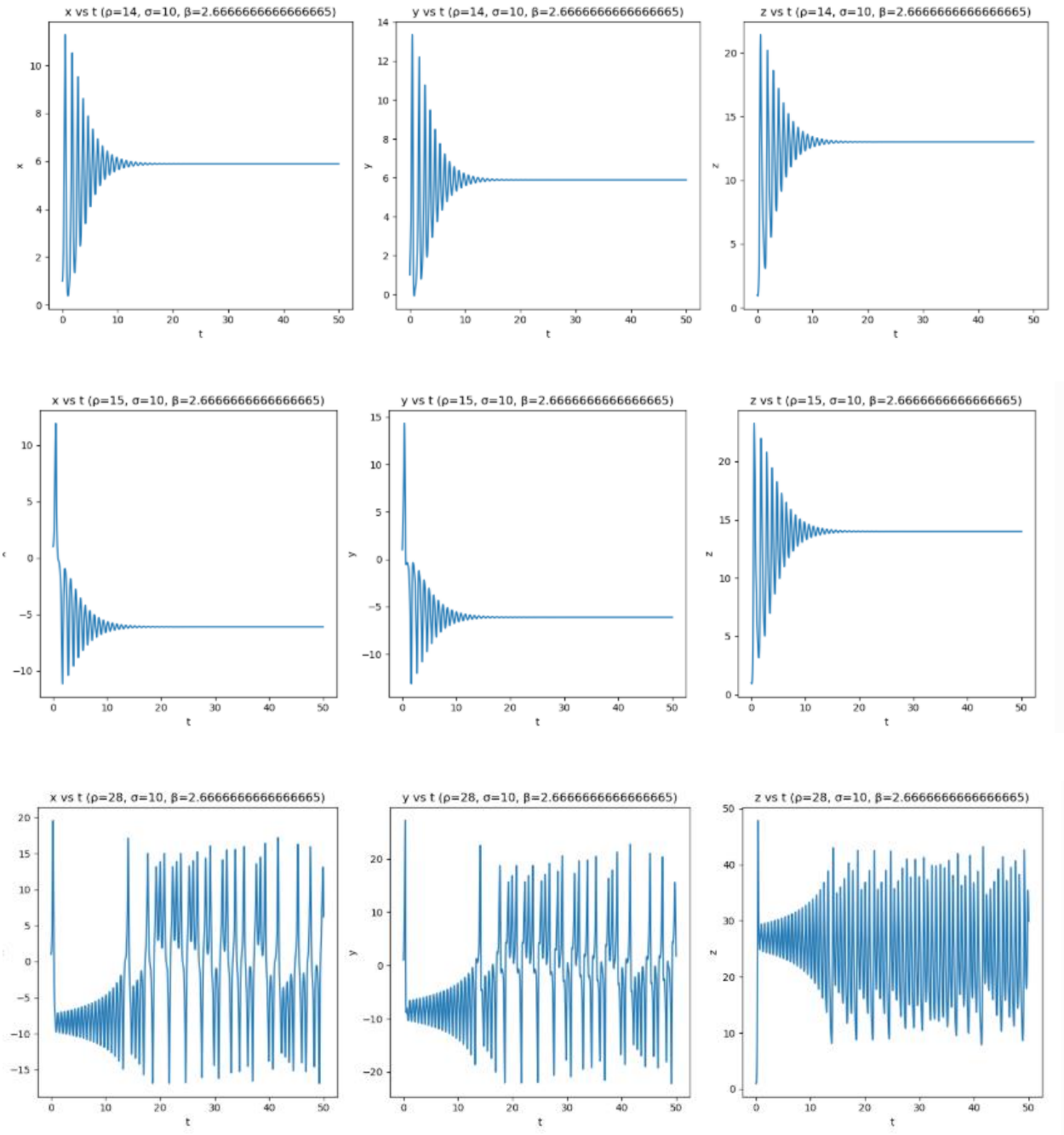

Figure 2.7.4: Lorentz system graph of phase space diagrams when changing initial conditions

2.7.5 Discussion

The 3D phase space diagrams clearly illustrate how the system shifts from simple, regular loops to increasingly complex and chaotic behavior as the value of ρ rises. At lower ρ values, the system shows symmetric, spiral-like patterns, indicating damped or periodic motion. However, when ρ reaches 28, it transitions into the well-known Lorenz attractor a strange attractor with a butterfly-like shape that is non-repeating, non-intersecting, yet bounded. This complexity is also visible in the 2D phase space (x-y and y-z) plots, which display dense, organized loops reflecting chaotic yet confined behavior.

A comparison of two simulations with almost identical initial conditions highlights the system's extreme sensitivity trajectories that begin similarly soon diverge significantly, a hallmark of chaos. The time-series plots of x(t), y(t), and z(t) further support this, showing irregular but bounded oscillations. These results emphasize the Lorenz system’s dual nature: it is deterministic and rule-based, yet practically unpredictable due to its sensitivity to initial states.

2.8. Exercise 8 (Sensitivity to initial conditions)

2.8.1 Objective

The goal of this experiment is to dive into the fascinating sensitivity to initial conditions that defines the chaotic nature of the Lorenz system. Even though these equations follow a predictable pattern, tiny tweaks in the starting point can lead to wildly different outcomes over time. In this study, we'll run simulations with almost identical initial states, changing only the x-value by a small amount, to show how these subtle differences can snowball into major divergences. By plotting x over time for these various starting points, we hope to capture the unpredictable and intricate beauty that makes chaotic systems so intriguing.

2.8.2 The code developed

```
from scipy.integrate import odeint
import numpy as np
import matplotlib.pyplot as plt

def lorentz(variables, t, p, s, b):
    x, y, z = variables
    dxdt = s * (y - x)
    dydt = x * (p - z) - y
    dzdt = x * y - b * z
    return [dxdt, dydt, dzdt]

t = np.linspace(0, 30, 10000)

init1 = [0.0, 1.0, 1.0]
init2 = [0.01, 1.0, 1.0]
init3 = [0.02, 1.0, 1.0]

sol1 = odeint(lorentz, init1, t, args=(70, 10, 8/3))
sol2 = odeint(lorentz, init2, t, args=(70, 10, 8/3))
sol3 = odeint(lorentz, init3, t, args=(70, 10, 8/3))

x1 = sol1[:, 0]
x2 = sol2[:, 0]
x3 = sol3[:, 0]

plt.figure(figsize=(15, 6))
plt.plot(t, x1, label='init1: [0.0, 1.0, 1.0]', color='red')
plt.plot(t, x2, label='init2: [0.01, 1.0, 1.0]', color='blue')
plt.plot(t, x3, label='init3: [0.02, 1.0, 1.0]', color='green')
plt.title('x(t) for Varying Initial Conditions - 09:59 PM +0530, Tuesday, August 05, 2025')
plt.xlabel('t')
plt.ylabel('x')
plt.legend()
plt.grid(True)
plt.show()
```

Figure 2.8.1: The code for plot that includes different initial conditions.

2.8.3 How it works

The provided Python code simulates the Lorenz system, a set of nonlinear differential equations known for exhibiting chaotic behavior, to demonstrate sensitivity to initial conditions. It uses `scipy.integrate.odeint` for numerical integration, `numpy` for array operations, and `matplotlib.pyplot` for visualization. The system is solved numerically using the odeint function from SciPy over a time range from 0 to 30 seconds. Multiple initial conditions are used, with small variations only in the x-component (e.g., x=0.0, 0.001, and 0.01) while keeping y=1.0 and z=1.0 fixed. The solutions for each case are plotted as x(t) over time, allowing for visual comparison of how trajectories evolve from nearly identical starting points.

2.8.4 Results

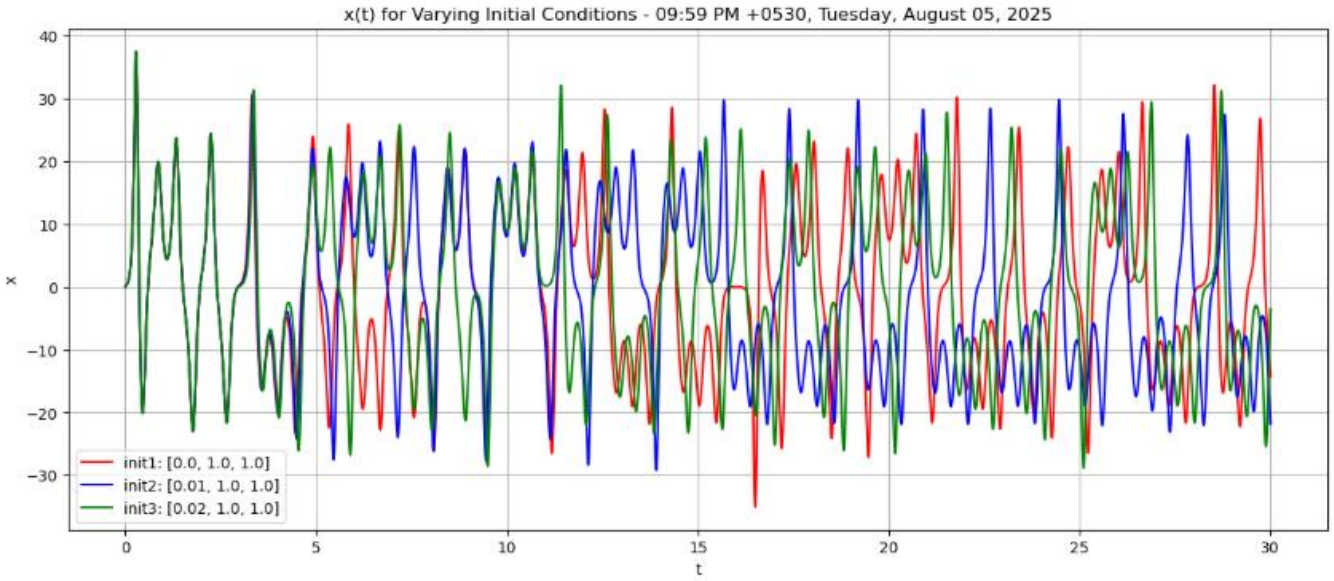

Figure 2.8.2: The plot that includes different initial conditions. Clearly it shows a difference between them.

2.8.5 Discussion

The resulting plot clearly demonstrates that the three trajectories, though starting nearly identically, begin to diverge significantly after a short time. Initially, they stay close, but even tiny differences in starting values—as small as 0.001 or 0.0000001 lead to completely different outcomes as time goes on. This illustrates sensitive dependence on initial conditions, a hallmark of chaotic systems. The divergence isn't caused by randomness but by the Lorenz system's nonlinear feedback, which causes small differences to grow exponentially. This confirms that while the system is deterministic, its long-term behavior is inherently unpredictable. Such behavior explains the difficulty of accurately modeling real-world chaotic systems, like weather patterns, where tiny measurement errors can lead to vastly different forecasts. The experiment clearly shows how chaos arises from deterministic rules and emphasizes the importance of precise initial conditions in understanding and predicting nonlinear dynamical systems.

2.9. Exercise 9 (Stable Points)

2.9.1 Objective

This experiment aims to explore the behavior of the Lorenz system using a specific set of parameters that result in non-chaotic dynamics. By setting $\rho = 10$, $\sigma = 10$, and $\beta = 8/3$, the system moves from chaotic motion to a stable and predictable state. Under these conditions, the Lorenz system is known to have three equilibrium points: one at the origin and two symmetrically located in phase space around $(x, y) \approx (\pm 5, \pm 5)$. The purpose of the experiment is to visualize trajectories starting from different initial conditions, confirm the positions of these stable points, and observe how the system's solutions evolve and settle toward them over time.

2.9.2 The code developed

```
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt

def lorenz_system(state, t, sigma, rho, beta):
    x, y, z = state
    dxdt = sigma * (y - x)
    dydt = x * (rho - z) - y
    dzdt = x * y - beta * z
    return [dxdt, dydt, dzdt]

initial_states = [[1.0, 0.1, 0.1], [5, 5, 5], [-5, -5, 5], [3.5, 3.5, 8.0], [-3.5, 3.5, 8.0],
                  [7, 7, 7], [-7, -7, 7]
]

time = np.linspace(0, 50, 10000)
sigma = 10
rho = 10
beta = 8 / 3

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
```

```
for state in initial_states:
    sol = odeint(lorenz_system, state, time, args=(sigma, rho, beta))
    x, y, z = sol.T
    ax.plot(x, y, z, label=f'Initial = {state}')

equil = np.sqrt(beta * (rho - 1))
equilibria = [[0, 0, 0], [equil, equil, rho - 1], [-equil, -equil, rho - 1]]

for eq in equilibria:
    ax.scatter(*eq, s=50, label=f'{tuple(np.round(eq, 2))}')

ax.set_title('Stable points in Lorenz system')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.legend()
plt.tight_layout()
plt.show()
```

Figure 2.9.1: The code that includes different initial conditions for Lorentz system.

2.9.3 How it works

This code simulates and visualizes the Lorenz system under non-chaotic conditions using multiple initial states. It defines the Lorenz system's differential equations with parameters $\sigma = 10$, $\rho = 10$, and $\beta = 8/3$, which result in stable behavior. The odeint function numerically solves the system for each of the seven given initial conditions over time. The 3D trajectories of each solution are plotted to show how they evolve and converge toward equilibrium points. Additionally, the three equilibrium points (origin and two symmetric points) are calculated and marked on the plot using scatter points. The result is a 3D visualization that demonstrates how the Lorenz system stabilizes to fixed points when non-chaotic parameters are used. The code highlights the system's predictable and stable nature under specific conditions.

2.9.4 Results

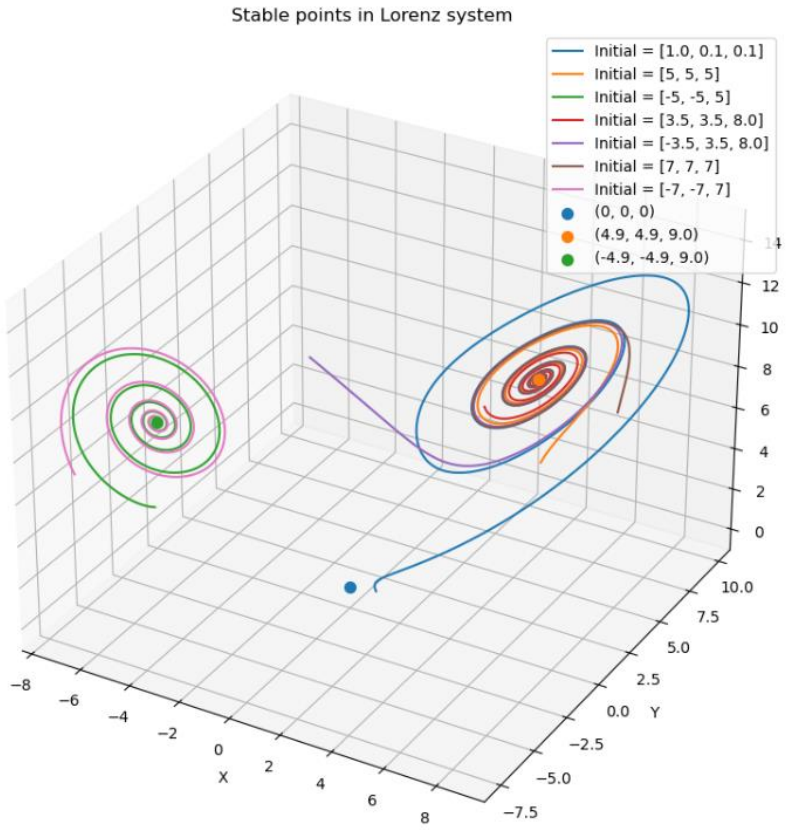

Figure 2.9.2: The plot that includes different initial conditions of stable points in Lorentz system. Clearly it shows a difference between them.

2.9.5 Discussion

The phase space plots show that the Lorenz system behaves predictably under the chosen parameters, with all trajectories settling at one of three stable equilibrium points. Near the origin, paths spiral inward and stop, confirming it as a stable point. The other two fixed points, around (±4.9, ±4.9, 9), attract nearby trajectories. This convergence is smooth and not sensitive to small initial changes, unlike chaotic behavior where small variations cause major divergence. These results demonstrate that, with $\rho = 10$, the system is stable and predictable. The experiment highlights how changing system parameters can shift dynamics from chaos to order.

2.10. Exercise 10 (Duffing Equation)

2.10.1 Objective

This experiment aims to explore the behavior of the Duffing oscillator a well-known nonlinear dynamical system under varying levels of external forcing. The focus is on examining how changes in the driving amplitude (γ) influence the system's displacement over time. By simulating the Duffing equation for different γ values and analyzing the resulting displacement-time plots, the study seeks to observe how the motion evolves, particularly noting transitions from simple, regular oscillations to more complex and possibly chaotic dynamics.

2.10.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

delta = 0.3
alpha = -1
beta = 1
gamma = [0.20,0.28,0.29,0.37, 0.5, 0.65]

gamma = [0.2, 0.5, 1.0, 1.5]
omega = 1.2

def duffing(t, y, alpha, beta, gamma, omega, delta):
    x, v = y
    dxdt = v
    dvdt = -delta * v - alpha * x - beta * x**3 + gamma * np.cos(omega * t)
    return [dxdt, dvdt]

y0 = [1.0, 0.0]
t_span = (0, 100)
t_eval = np.linspace(t_span[0], t_span[1], 5000)

plt.figure(figsize=(10, 8))

for gam in gamma:
    solution = solve_ivp(
        lambda t, y: duffing(t, y, alpha, beta, gam, omega, delta),
        t_span,
        y0,
        t_eval=t_eval
    )

    x = solution.y[0]
    v = solution.y[1]

    plt.plot(x, v, label=f'Gamma = {gam}')

plt.title("Duffing Oscillator with Negative Damping")
plt.xlabel("Displacement (x)")
plt.ylabel("Velocity (v)")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

Figure 2.10.1: The code that includes different gamma values for duffing equation.

2.10.3 How it works

The provided Python code simulates a Duffing oscillator with negative damping. It uses scipy.integrate.solve_ivp, a robust numerical solver, to solve the system's first-order differential equations. The core of the simulation is the duffing function, which defines the system's dynamics, including a negative damping term (delta = -0.3), a nonlinear restoring force, and an external periodic forcing term. The code runs this simulation for various forcing amplitudes (gamma). For each gamma value, it plots the resulting trajectory on a single-phase space diagram (velocity vs. position). This visualization allows for a direct comparison of how the system's long-term behavior is influenced by the strength of the external forcing, especially when the inherent negative damping is driving the system to gain energy.

2.10.4 Results

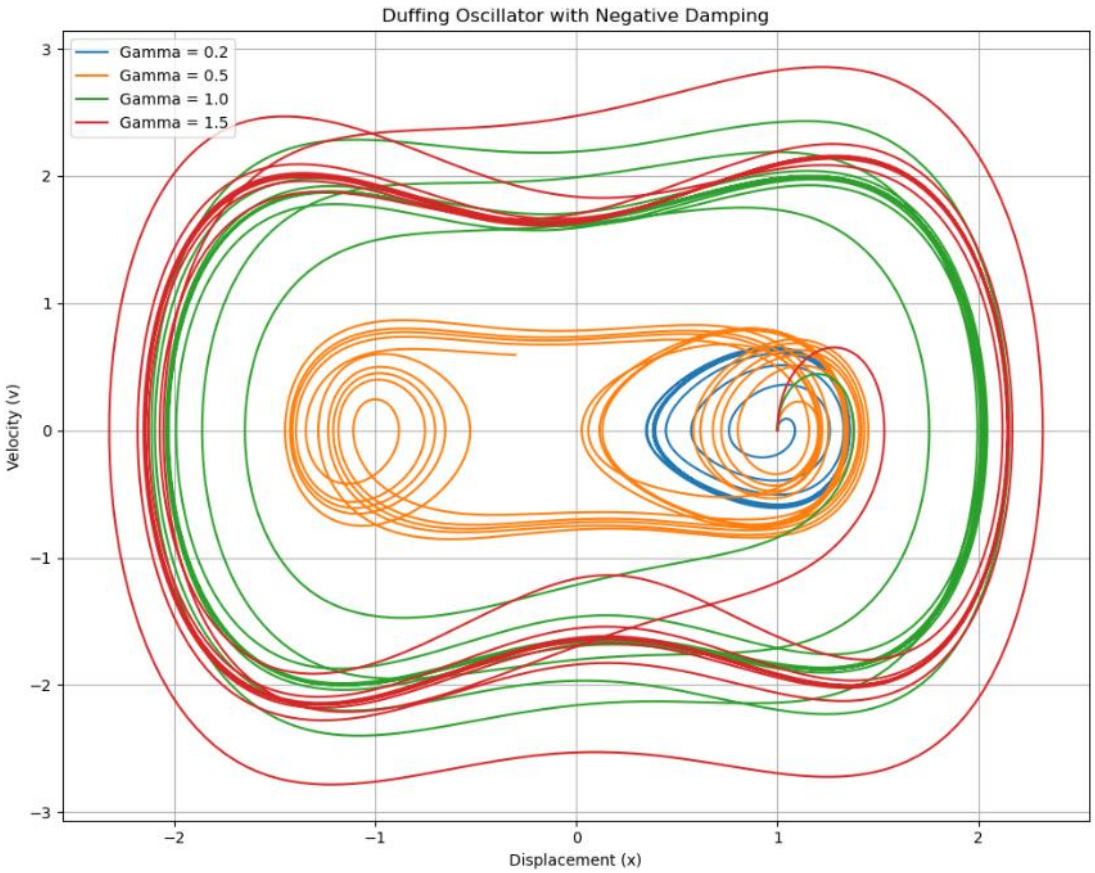

Figure 2.10.2: The plots that includes different gamma values

2.10.5 Discussion

The time-domain plots show that the Duffing oscillator's behavior changes significantly with increasing forcing amplitude, γ. At low values, such as 0.20 and 0.28, the system exhibits simple, predictable periodic motion. However, as γ is increased to around 0.29, the oscillations become more intricate and lose their regular pattern. For higher forcing amplitudes like 0.50 and 0.65, the motion becomes aperiodic and highly irregular, signifying the emergence of chaos. This transition from stable, periodic behavior to chaos through a minor change in a system parameter is a fundamental feature of nonlinear dynamics and is effectively demonstrated by this experiment.

3. Conclusion

This series of numerical experiments provided a thorough investigation into the behavior of both linear and nonlinear dynamic systems. Beginning with damped pendulum models, we saw how modifying parameters like damping and forcing alters system stability and periodicity, observing transitions between different equilibrium types. The Van der Pol oscillator then demonstrated how nonlinear damping can create a stable, self-sustaining oscillation known as a limit cycle. Through the Duffing oscillator, we explored how an increasing external force can transition a system from regular, predictable motion to chaotic behavior, showcasing nonlinear resonance and bifurcation.

The Lorenz system was a powerful tool for introducing chaotic dynamics, clearly showing how even tiny changes in starting conditions lead to drastically different outcomes. This sensitivity was visually confirmed by comparing trajectories and time-series plots. To further analyze this chaos, we computed the Lyapunov exponents and Kaplan–Yorke dimension. The existence of a positive exponent verified the chaotic nature of the system, while the dimension falling between 2 and 3 confirmed the presence of a strange attractor, a fractal structure representing bounded but aperiodic motion.

Overall, this process successfully illustrated how nonlinear systems can stabilize or become chaotic. These exercises highlight the complex balance between order and unpredictability in dynamic systems and deepen our understanding of the mathematical concepts that govern their behavior.